

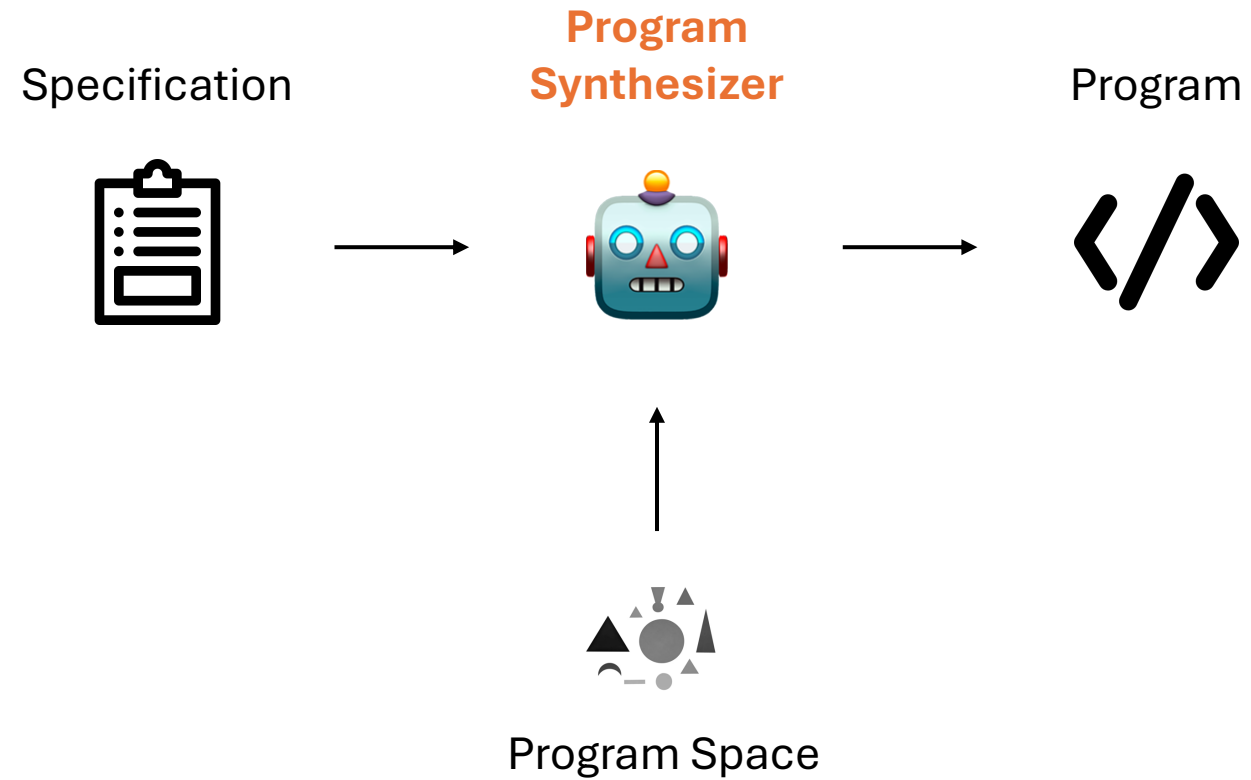
Machine Programming

Lecture 2 – Syntax, Semantics, and Inductive Synthesis

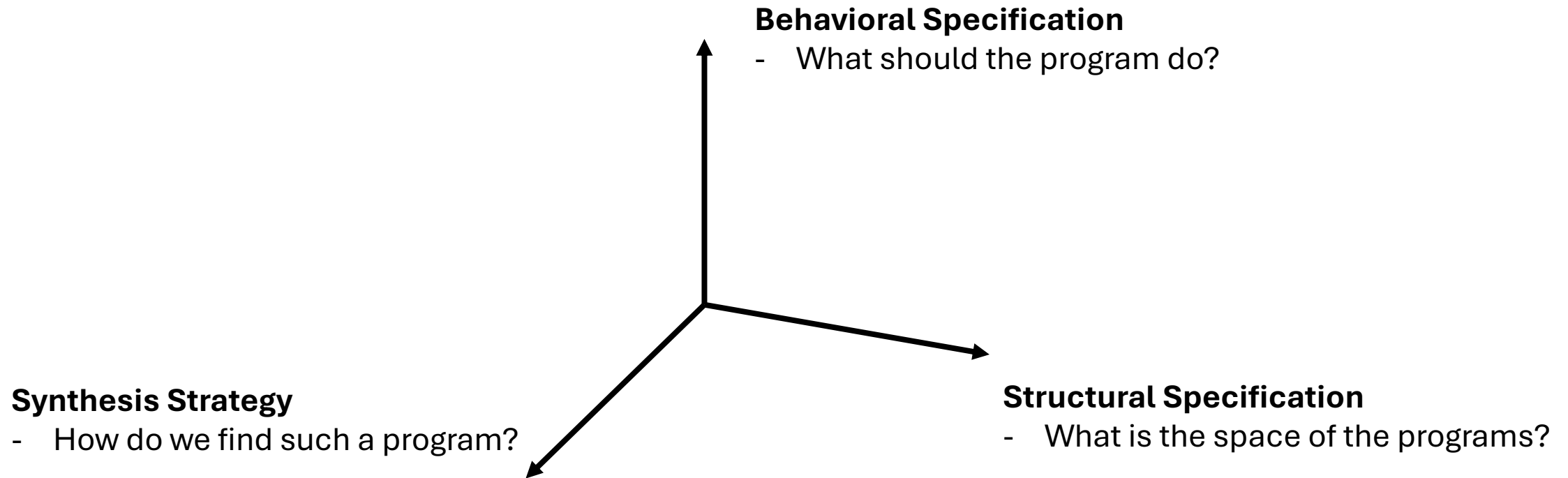
Ziyang Li

Program Synthesis

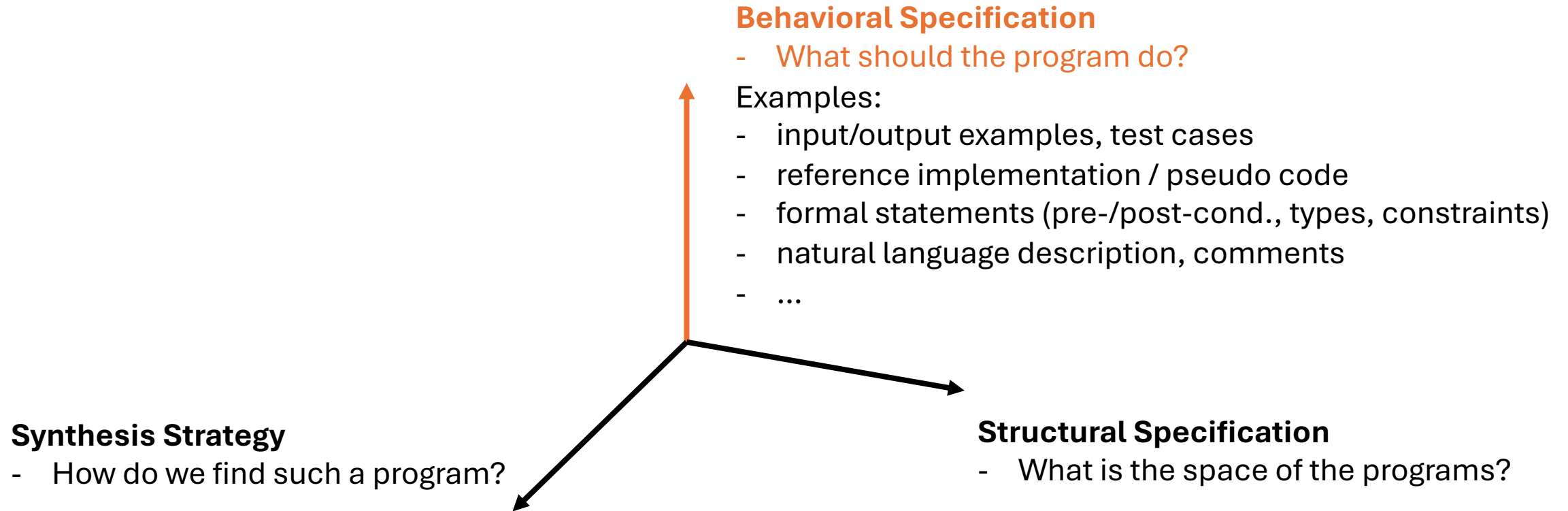
Program Synthesizers



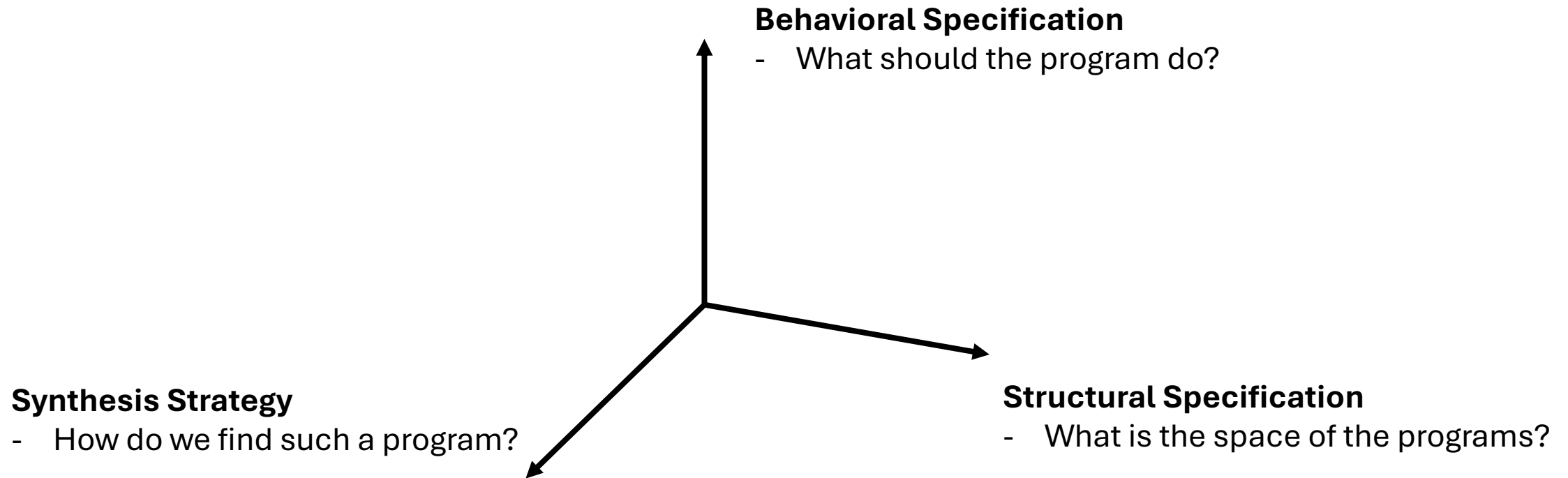
Dimensions in Program Synthesis



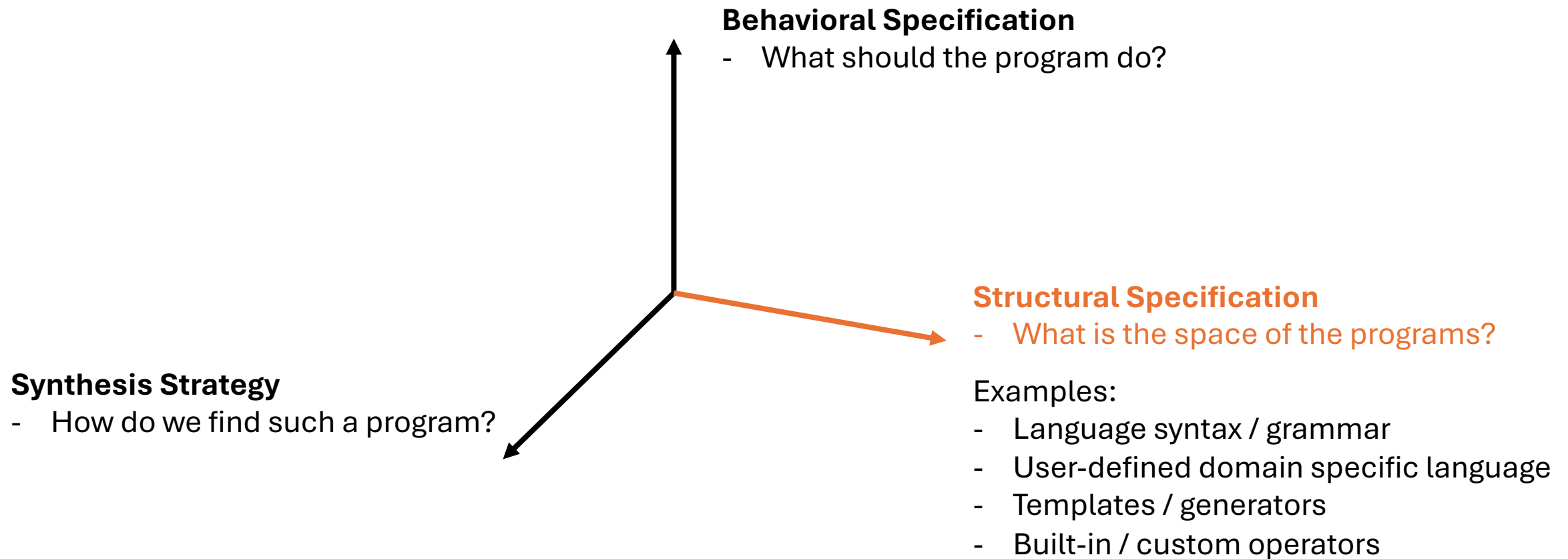
Dimensions in Program Synthesis



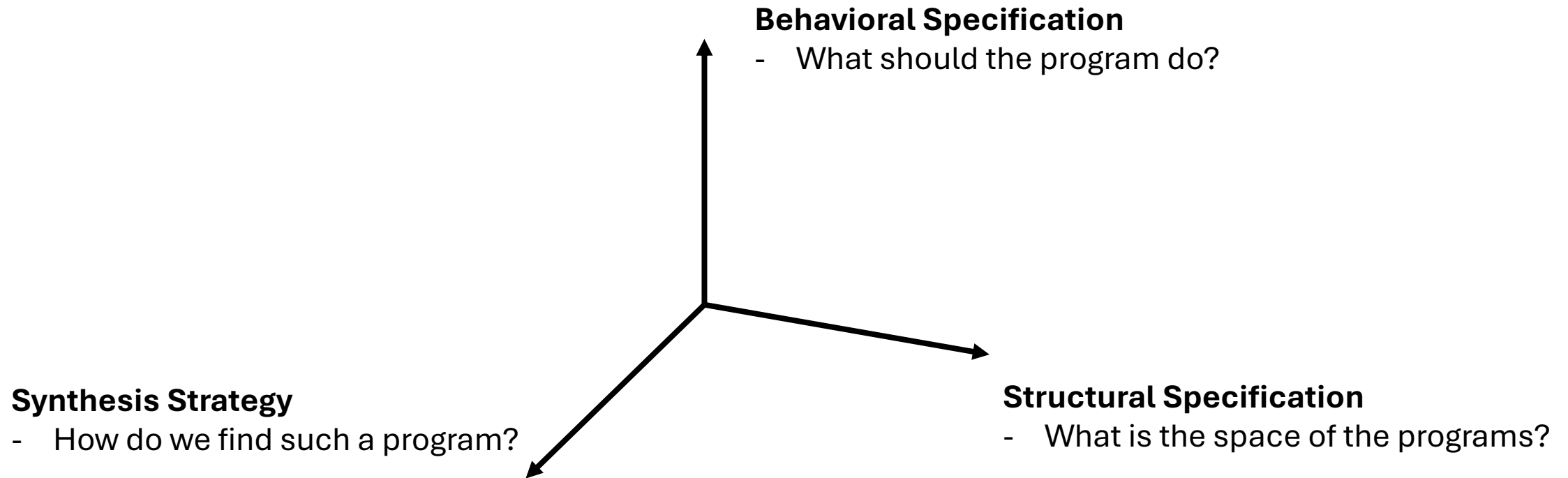
Dimensions in Program Synthesis



Dimensions in Program Synthesis



Dimensions in Program Synthesis



Dimensions in Program Synthesis

Behavioral Specification

- What should the program do?

Synthesis Strategy

- How do we find such a program?

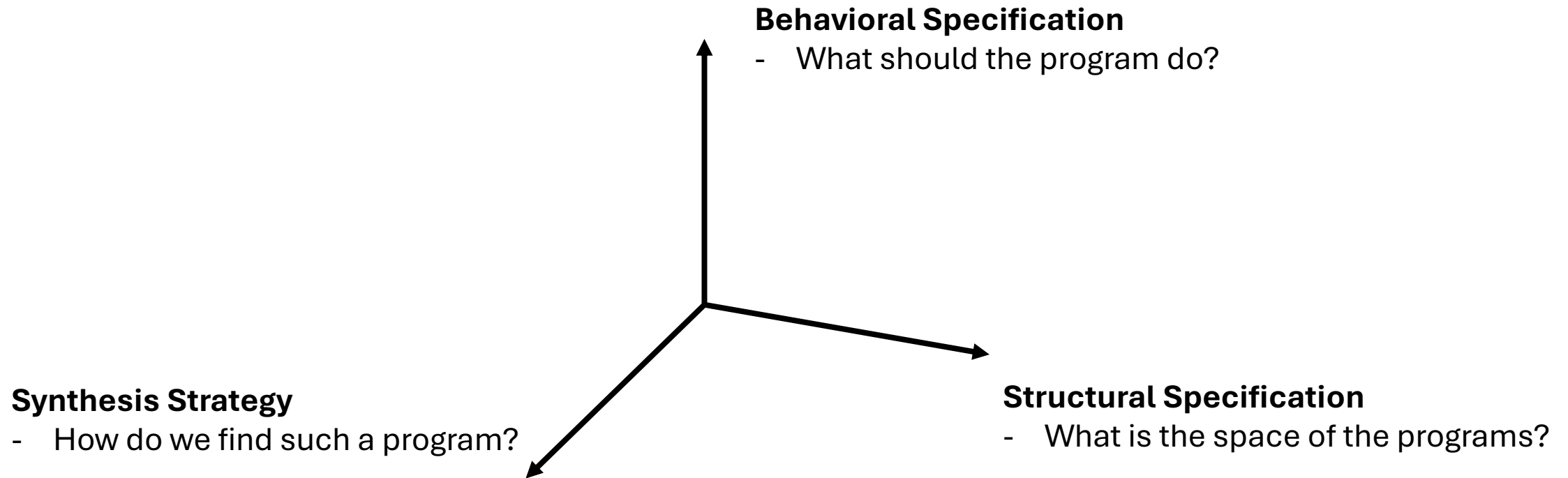
Examples:

- Bottom-up / top-down search
- Stochastic / constraint-based search
- LLM zero-shot / few-shot prompting
- Chain-of-thought reasoning
- Iterative / agentic self-refinement

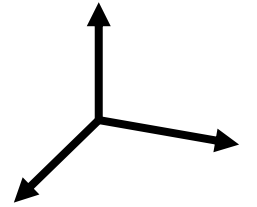
Structural Specification

- What is the space of the programs?

Dimensions in Program Synthesis



Program Synthesis in Action



Behavioral Specification

- What should the program do?

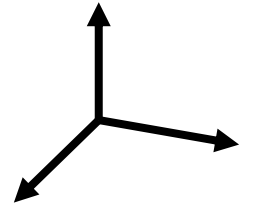
Structural Specification

- What is the space of the programs?

Synthesis Strategy

- How do we find such a program?

Program Synthesis in Action



Behavioral Specification

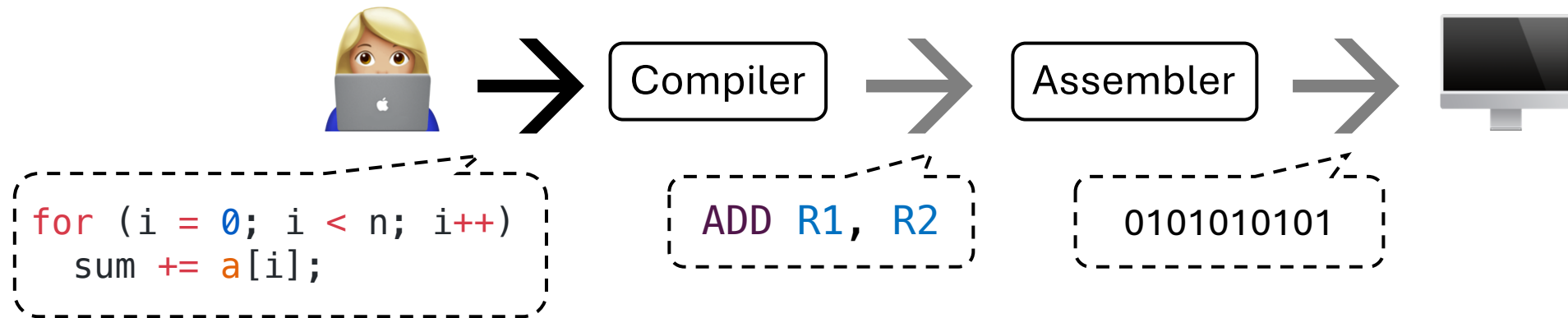
- What should the program do?

Structural Specification

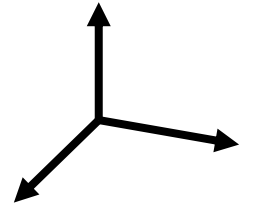
- What is the space of the programs?

Synthesis Strategy

- How do we find such a program?



Program Synthesis in Action



Behavioral Specification

- What should the program do?

A program in high-level programming language (Fortran, C, Rust, etc.)

Structural Specification

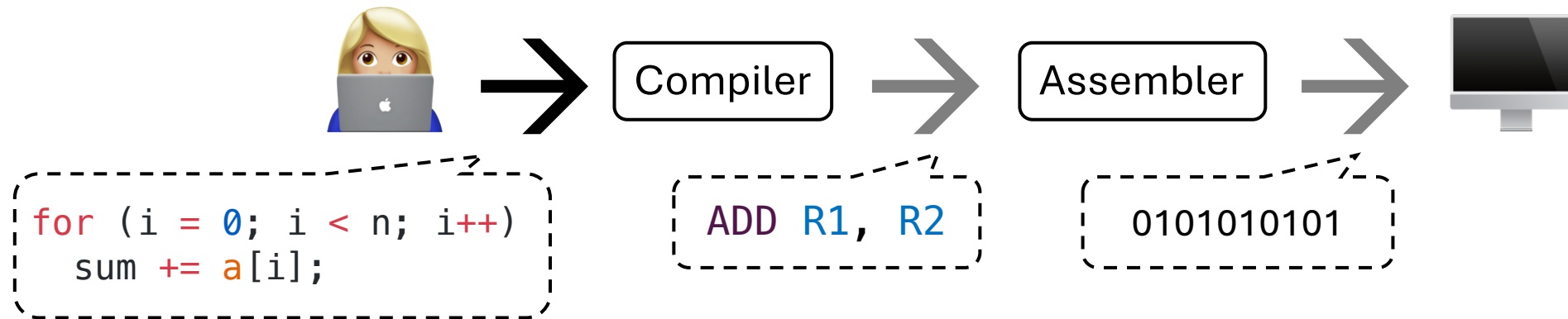
- What is the space of the programs?

A program in binary machine code

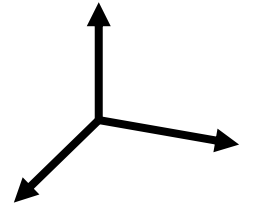
Synthesis Strategy

- How do we find such a program?

Compilation, Optimization, Assembling, etc.



Program Synthesis in Action



Behavioral Specification

- What should the program do?

Structural Specification

- What is the space of the programs?

Synthesis Strategy

- How do we find such a program?

Quick Fill

Auto Fill

Quick Layout

Apply

HiLight

CurrencyWidget

Undo

Commit

AddressWidget

Table116	Ana Trujillo 357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171					
	A	B	C	D	E	F
1	Column1	Col 2	Col 3	Col 4	Col 5	Col 6
2	Ana Trujillo	357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171	Redmond	WA	(757) 555-1634	140-37-6064 27171
3	Antonio Moreno	515 93th Lane ,Renton,WA,(411) 555-2786,562-87-3127,28581				
4	Thomas Hardy	742 17th Street NE,Seattle,WA,(412) 555-5719,921-29-4931,24607				
5	Christina Berglund	475 22th Lane ,Redmond,WA,(443) 555-6774,844-35-6764,30146				
6	Hanna Moos	785 45th Street NE,Puyallup,WA,(376) 555-2462,515-68-1285,29284				
7	Frédérique Citeaux	308 66th Place ,Redmond,WA,(689) 555-2770,552-23-2508,21415				
8	Martin Sommer	887 86th Place ,Kent,WA,(715) 555-5450,870-91-9824,21536				
9	Laurence Lebihan	944 13th Street NE,Redmond,WA,(620) 555-2361,649-25-5312,25252				
10	Elizabeth Lincoln	452 73th Lane NE,Renton,WA,(851) 555-4561,425-97-6344,22279				
11	Victoria Ashworth	463 16th Street ,Renton,WA,(696) 555-6044,690-29-7926,22832				
12	Patricio Simpson	630 20th Street ,Redmond,WA,(179) 555-3265,389-78-3236,24525				
13	Francisco Chang	683 49th Lane ,Seattle,WA,(272) 555-7434,665-18-6435,29453				
14	Yang Wang	944 28th Lane ,Redmond,WA,(151) 555-2272,846-78-8452,24388				
15	Pedro Afonso	411 70th Place ,Kent,WA,(170) 555-2964,774-35-2298,29485				
16	Elizabeth Brown	971 20th Lane ,Puyallup,WA,(373) 555-4134,476-53-7164,26417				
17	Sven Ottlieb	676 17th Lane NE,Redmond,WA,(828) 555-1593,548-73-8633,27440				
18	Janine Labrune	267 95th Place SE,Seattle,WA,(949) 555-1316,350-27-8300,28074				
19	Ann Devon	694 53th Place ,Kent,WA,(194) 555-8124,559-74-4016,22367				
20	Roland Mendel	581 12th Street NW,Kent,WA,(103) 555-2146,303-79-1328,20518				
21	Aria Cruz	594 85th Lane ,Renton,WA,(431) 555-1376,329-93-9992,21498				
22	Diego Roel	550 22th Lane ,Renton,WA,(639) 555-6238,918-34-5172,25931				
23	Martine Rancé	688 93th Place NW,Kent,WA,(573) 555-3571,695-94-3479,22424				
24						
25						
26						

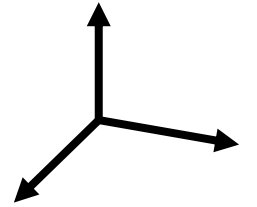
Quick Fill Auto Fill Quick Layout

Apply HiLight CurrencyWidget

Undo Commit AddressWidget

Table116 Ana Trujillo 357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171								
	A	B	C	D	E	F		
1	Column1	Col 2	Col 3	Col 4	Col 5	Col 6		
2	Ana Trujillo 357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171	Redmond	WA	(757) 555-1634	140-37-6064	27171		
3	Antonio Moreno 515 93th Lane ,Renton,WA,(411) 555-2786,562-87-3127,28581	Renton	WA	(411) 555-2786	562-87-3127	28581		
4	Thomas Hardy 742 17th Street NE,Seattle,WA,(412) 555-5719,921-29-4931,24607	Seattle	WA	(412) 555-5719	921-29-4931	24607		
5	Christina Berglund 475 22th Lane ,Redmond,WA,(443) 555-6774,844-35-6764,30146	Redmond	WA	(443) 555-6774	844-35-6764	30146		
6	Hanna Moos 785 45th Street NE,Puyallup,WA,(376) 555-2462,515-68-1285,29284	Puyallup	WA	(376) 555-2462	515-68-1285	29284		
7	Frédérique Citeaux 308 66th Place ,Redmond,WA,(689) 555-2770,552-23-2508,21415	Redmond	WA	(689) 555-2770	552-23-2508	21415		
8	Martin Sommer 887 86th Place ,Kent,WA,(715) 555-5450,870-91-9824,21536	Kent	WA	(715) 555-5450	870-91-9824	21536		
9	Laurence Lebihan 944 13th Street NE,Redmond,WA,(620) 555-2361,649-25-5312,2525	Redmond	WA	(620) 555-2361	649-25-5312	25252		
10	Elizabeth Lincoln 452 73th Lane NE,Renton,WA,(851) 555-4561,425-97-6344,22279	Renton	WA	(851) 555-4561	425-97-6344	22279		
11	Victoria Ashworth 463 16th Street ,Renton,WA,(696) 555-6044,690-29-7926,22832	Renton	WA	(696) 555-6044	690-29-7926	22832		
12	Patricio Simpson 630 20th Street ,Redmond,WA,(179) 555-3265,389-78-3236,24525	Redmond	WA	(179) 555-3265	389-78-3236	24525		
13	Francisco Chang 683 49th Lane ,Seattle,WA,(272) 555-7434,665-18-6435,29453	Seattle	WA	(272) 555-7434	665-18-6435	29453		
14	Yang Wang 944 28th Lane ,Redmond,WA,(151) 555-2272,846-78-8452,24388	Redmond	WA	(151) 555-2272	846-78-8452	24388		
15	Pedro Afonso 411 70th Place ,Kent,WA,(170) 555-2964,774-35-2298,29485	Kent	WA	(170) 555-2964	774-35-2298	29485		
16	Elizabeth Brown 971 20th Lane ,Puyallup,WA,(373) 555-4134,476-53-7164,26417	Puyallup	WA	(373) 555-4134	476-53-7164	26417		
17	Sven Ottlieb 676 17th Lane NE,Redmond,WA,(828) 555-1593,548-73-8633,27440	Redmond	WA	(828) 555-1593	548-73-8633	27440		
18	Janine Labrune 267 95th Place SE,Seattle,WA,(949) 555-1316,350-27-8300,28074	Seattle	WA	(949) 555-1316	350-27-8300	28074		
19	Ann Devon 694 53th Place ,Kent,WA,(194) 555-8124,559-74-4016,22367	Kent	WA	(194) 555-8124	559-74-4016	22367		
20	Roland Mendel 581 12th Street NW,Kent,WA,(103) 555-2146,303-79-1328,20518	Kent	WA	(103) 555-2146	303-79-1328	20518		
21	Aria Cruz 594 85th Lane ,Renton,WA,(431) 555-1376,329-93-9992,21498	Renton	WA	(431) 555-1376	329-93-9992	21498		
22	Diego Roel 550 22th Lane ,Renton,WA,(639) 555-6238,918-34-5172,25931	Renton	WA	(639) 555-6238	918-34-5172	25931		
23	Martine Rancé 688 93th Place NW,Kent,WA,(573) 555-3571,695-94-3479,22424	Kent	WA	(573) 555-3571	695-94-3479	22424		
24								
25								
26								

Program Synthesis in Action



Behavioral Specification

- What should the program do?

Fit the user demonstrated
example rows

Structural Specification

- What is the space of the programs?

Excel formula language

Synthesis Strategy

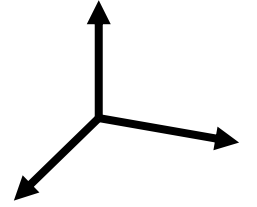
- How do we find such a program?

Programming by
demonstration (PbD)

Column1	Column2	Column3	Column4	Column5	Column6
Ana Trujillo	357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171	Redmond	WA	(757) 555-1634	140-37-6064 27171
Antonio Moreno	515 93th Lane ,Renton,WA,(411) 555-2786,562-87-3127,28581	Renton	WA	(411) 555-2786	562-87-3127 28581
Thomas Hardy	742 17th Street NE,Seattle,WA,(412) 555-5719,921-29-4931,24607	Seattle	WA	(412) 555-5719	921-29-4931 24607
Christina Berglund	475 22th Lane ,Redmond,WA,(443) 555-6774,844-35-6764,30146	Redmond	WA	(443) 555-6774	844-35-6764 30146
Hanna Moos	785 45th Street NE,Puyallup,WA,(376) 555-2462,515-68-1285,29284	Puyallup	WA	(376) 555-2462	515-68-1285 29284
Frédérique Citeaux	308 66th Place ,Redmond,WA,(689) 555-2770,552-23-2508,21415	Redmond	WA	(689) 555-2770	552-23-2508 21415
Martin Sommer	887 86th Place ,Kent,WA,(715) 555-5450,870-91-9824,21536	Kent	WA	(715) 555-5450	870-91-9824 21536
Laurence Leblhan	944 13th Street NE,Redmond,WA,(620) 555-2361,649-25-5312,25252	Redmond	WA	(620) 555-2361	649-25-5312 25252
Elizabeth Lincoln	452 73th Lane NE,Renton,WA,(851) 555-4561,425-97-6344,22279	Renton	WA	(851) 555-4561	425-97-6344 22279
Victoria Ashworth	463 16th Street ,Renton,WA,(696) 555-6044,690-29-7926,22832	Renton	WA	(696) 555-6044	690-29-7926 22832
Patricio Simpson	630 20th Street ,Redmond,WA,(179) 555-3265,389-78-3236,24525	Redmond	WA	(179) 555-3265	389-78-3236 24525
Francisco Chang	683 49th Lane ,Seattle,WA,(272) 555-7434,665-18-6435,29453	Seattle	WA	(272) 555-7434	665-18-6435 29453
Yang Wang	944 28th Lane ,Redmond,WA,(151) 555-2272,846-78-8452,24388	Redmond	WA	(151) 555-2272	846-78-8452 24388
Pedro Afonso	411 70th Place ,Kent,WA,(170) 555-2964,774-35-2298,29485	Kent	WA	(170) 555-2964	774-35-2298 29485
Elizabeth Brown	971 20th Lane ,Puyallup,WA,(373) 555-4134,476-53-7164,26417	Puyallup	WA	(373) 555-4134	476-53-7164 26417
Sven Ottlieb	676 17th Lane NE,Redmond,WA,(828) 555-1593,548-73-8633,27440	Redmond	WA	(828) 555-1593	548-73-8633 27440
Janine Labrune	267 95th Place SE,Seattle,WA,(949) 555-1316,350-27-8300,28074	Seattle	WA	(949) 555-1316	350-27-8300 28074
Ann Devon	694 53th Place ,Kent,WA,(194) 555-8124,559-74-4016,22367	Kent	WA	(194) 555-8124	559-74-4016 22367
Roland Mendel	581 12th Street NW,Kent,WA,(103) 555-2146,303-79-1328,20518	Kent	WA	(103) 555-2146	303-79-1328 20518
Aria Cruz	594 85th Lane ,Renton,WA,(431) 555-1376,329-93-9992,21498	Renton	WA	(431) 555-1376	329-93-9992 21498
Diego Roel	550 22th Lane ,Renton,WA,(639) 555-6238,918-34-5172,25931	Renton	WA	(639) 555-6238	918-34-5172 25931
Martine Rancé	688 93th Place NW,Kent,WA,(573) 555-3571,695-94-3479,22424	Kent	WA	(573) 555-3571	695-94-3479 22424

FlashFill: a feature of Excel 2013 [Gulwani 2011]

Program Synthesis in Action



Behavioral Specification

- What should the program do?

Structural Specification

- What is the space of the programs?

Synthesis Strategy

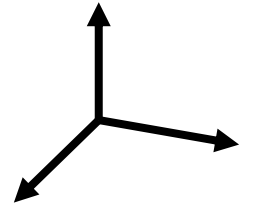
- How do we find such a program?

Github Copilot

```
// find all images
// and add a green border around them
// and add class "githubCopilot" to them
function run() {
```

```
    var images = document.getElementsByTagName('img');
    for (var i = 0; i < images.length; i++) {
        if (images[i].className.indexOf('githubCopilot') == -1) {
            images[i].className += ' githubCopilot';
            images[i].style.border = '1px solid green';
        }
    }
}
```

Program Synthesis in Action



Behavioral Specification

- What should the program do?

Function signature; comment
in natural language

Structural Specification

- What is the space of the programs?

JavaScript used in Web

Synthesis Strategy

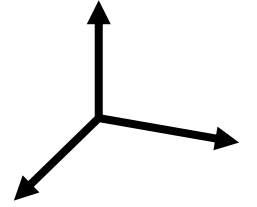
- How do we find such a program?

Large language model; next
token prediction

```
// find all images
// and add a green border around them
// and add class "githubCopilot" to them
function run() {
```

```
    var images = document.getElementsByTagName('img');
    for (var i = 0; i < images.length; i++) {
        if (images[i].className.indexOf('githubCopilot') == -1) {
            images[i].className += ' githubCopilot';
            images[i].style.border = '1px solid green';
        }
    }
}
```

Program Synthesis in Action



Behavioral Specification

- What should the program do?

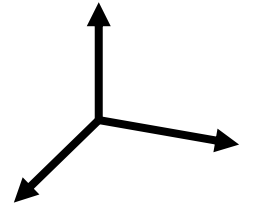
Structural Specification

- What is the space of the programs?

Synthesis Strategy

- How do we find such a program?

Program Synthesis in Action



Behavioral Specification

- What should the program do?

Description of animation,
functionalities in conversation
and markdown files

Structural Specification

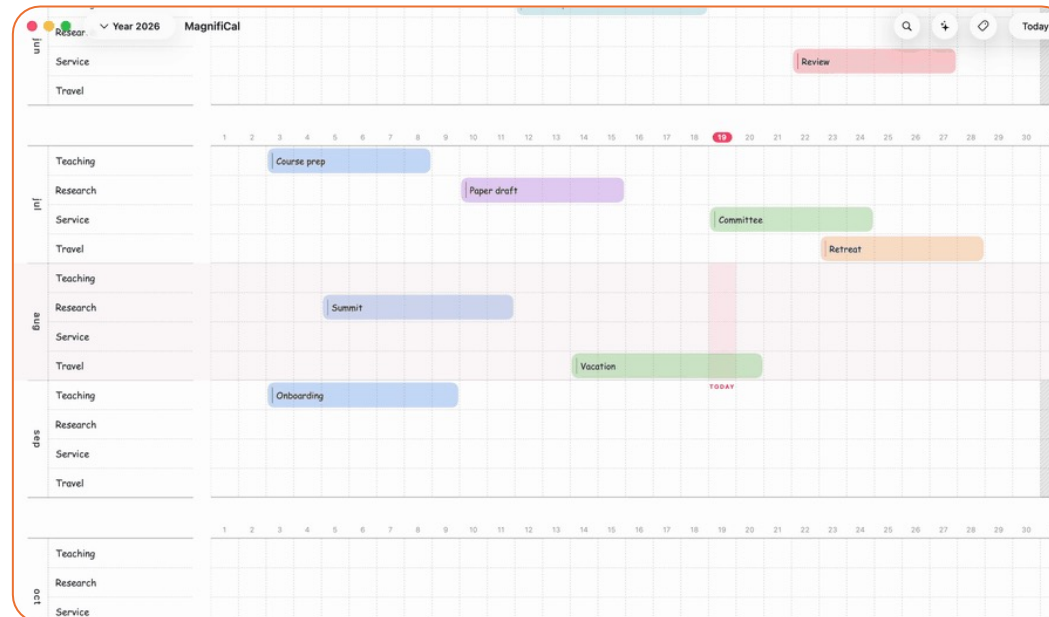
- What is the space of the programs?

A Swift program

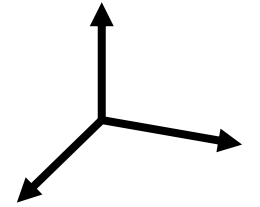
Synthesis Strategy

- How do we find such a program?

LLM-agent working with
Xcode and UI debugging
tools



Program Synthesis in Action



Behavioral Specification

- What should the program do?

Description of animation, functionalities in conversation and markdown files

Structural Specification

- What is the space of the programs?

A Swift program

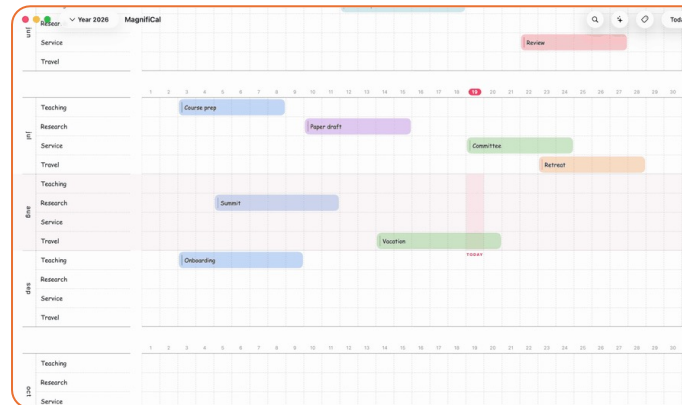
Synthesis Strategy

- How do we find such a program?

LLM-agent working with Xcode and UI debugging tools

```
func drawDrawerBehavior() {
    // (a) Define drawer behavior
    // A drawer will appear when I double click on an event. Currently, a huge drawer will be drawn out from the right
    // There is a current problem that, if I do not draw the main calendar canvas, if the event is towards the right hand
    // side of the screen, then the drawer will include that event; if I do shift the main calendar canvas to the left, then
    // if the event is on the left hand side of the canvas, the event will go out of canvas
    // The proposed solution is that, we should understand which event we are clicking on, and the goal is to shift
    // the main canvas to a point where the event is horizontally centered on the remaining space.
    // The computation is the following:
    // - Say the screen width is X
    // - Say the drawer's width is D, D < X
    // - When the drawer is drawn out, the left-hand-side available space is of width X - D
    // - Say the event is originally horizontally located at E < X (assume E is the left-hand-side)
    // - The goal would be shifting the main canvas left such that the new event location E' is in the center of X - D,
    // meaning (X - D) / 2 = E'
    // - Therefore, the left shift delta of the main canvas is D - (X - D) / 2
    // - We do want to cap the shift delta to be above zero, because we do not want the main canvas to shift
    // leftwards
    // - Meaning that if an event is already placed towards the left hand side of the screen and even to the left of
    // the sliding point in the first draw, we do not need to move the canvas.
    // A few notes:
    // - When we realize the drawer, the main canvas should be responsive and move accordingly.
    // - (c) Update: We also want to upper bound the drawer
    // - We do not want the shift delta to be larger than D: D - (X - D) / 2 > 0
}

// (d) Scroll-pano behavior in monthly view
// When we scroll up or down in the monthly view, we should be able to go to previous month and next month
// The current behavior is that in the monthly view, we cannot scroll, neither left/right nor up/down
// The scroll behavior should be the following:
// - First each month as if they are vertical pages
// - We are swiping across pages, like they are iPhone's homepages, just put vertical
// - When we swipe up from current month to next month, the action should be:
//   - The daily timeline view first fades out (without distortion)
//   - The next month's tracks come up from the bottom
// - When it gets closer to the top, the current month's track (originally stayed on the top) will move upwards
// and disappear out at the top
// - And the next month's daily view starts appearing (fade-in without distortion)
// - When we swipe down from current month to previous month, the action would be the mostly the reversed
// - The daily timeline view first fades out
// - The current month's track first moves downwards
// - While that is happening, the previous month's tracks come down from the top
// - The previous month's tracks disappear at the same place of the current month's tracks
// - The current month's tracks keep moving down and go out of the screen
// - The previous month's daily view starts appearing (fade-in)
```



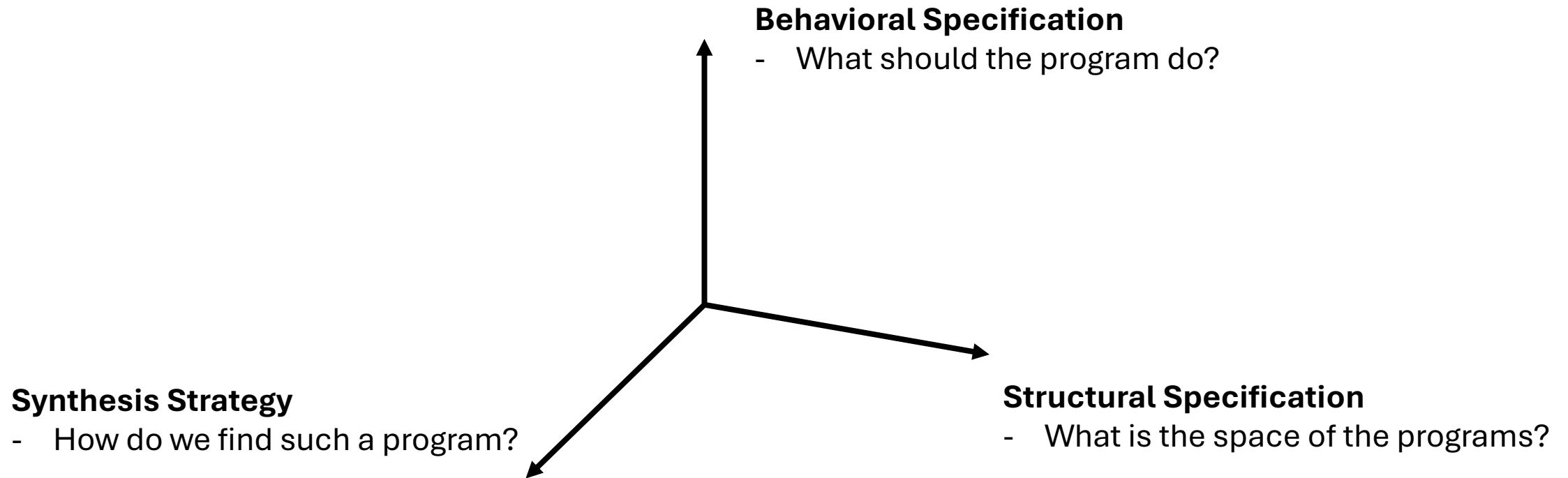
```
func drawDrawerBehavior() {
    // (a) Define drawer behavior
    // A drawer will appear when I double click on an event. Currently, a huge drawer will be drawn out from the right
    // There is a current problem that, if I do not draw the main calendar canvas, if the event is towards the right hand
    // side of the screen, then the drawer will include that event; if I do shift the main calendar canvas to the left, then
    // if the event is on the left hand side of the canvas, the event will go out of canvas
    // The proposed solution is that, we should understand which event we are clicking on, and the goal is to shift
    // the main canvas to a point where the event is horizontally centered on the remaining space.
    // The computation is the following:
    // - Say the screen width is X
    // - Say the drawer's width is D, D < X
    // - When the drawer is drawn out, the left-hand-side available space is of width X - D
    // - Say the event is originally horizontally located at E < X (assume E is the left-hand-side)
    // - The goal would be shifting the main canvas left such that the new event location E' is in the center of X - D,
    // meaning (X - D) / 2 = E'
    // - Therefore, the left shift delta of the main canvas is D - (X - D) / 2
    // - We do want to cap the shift delta to be above zero, because we do not want the main canvas to shift
    // leftwards
    // - Meaning that if an event is already placed towards the left hand side of the screen and even to the left of
    // the sliding point in the first draw, we do not need to move the canvas.
    // A few notes:
    // - When we realize the drawer, the main canvas should be responsive and move accordingly.
    // - (c) Update: We also want to upper bound the drawer
    // - We do not want the shift delta to be larger than D: D - (X - D) / 2 > 0
}

// (d) Scroll-pano behavior in monthly view
// When we scroll up or down in the monthly view, we should be able to go to previous month and next month
// The current behavior is that in the monthly view, we cannot scroll, neither left/right nor up/down
// The scroll behavior should be the following:
// - First each month as if they are vertical pages
// - We are swiping across pages, like they are iPhone's homepages, just put vertical
// - When we swipe up from current month to next month, the action should be:
//   - The daily timeline view first fades out (without distortion)
//   - The next month's tracks come up from the bottom
// - When it gets closer to the top, the current month's track (originally stayed on the top) will move upwards
// and disappear out at the top
// - And the next month's daily view starts appearing (fade-in without distortion)
// - When we swipe down from current month to previous month, the action would be the mostly the reversed
// - The daily timeline view first fades out
// - The current month's track first moves downwards
// - While that is happening, the previous month's tracks come down from the top
// - The previous month's tracks disappear at the same place of the current month's tracks
// - The current month's tracks keep moving down and go out of the screen
// - The previous month's daily view starts appearing (fade-in)
```

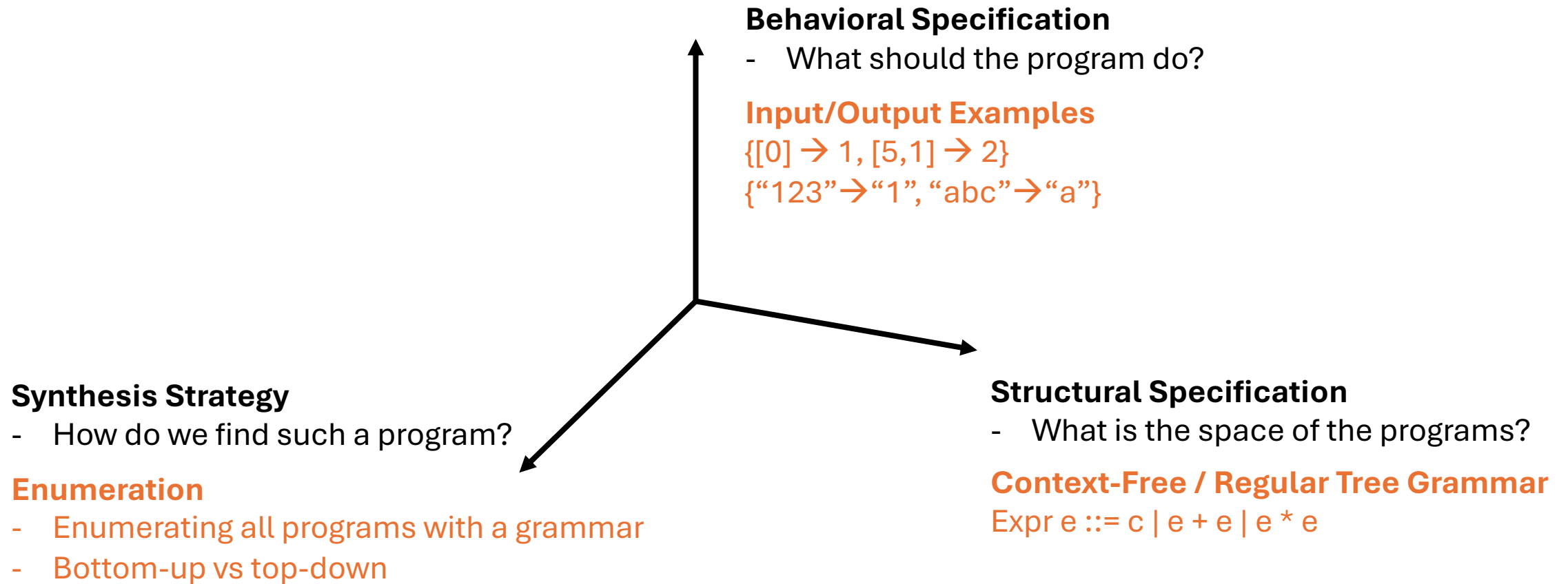
Structure of the course

- **Module 1: Foundations of Program Synthesis**
 - Programming language syntax and semantics
 - Classical methods for synthesizing programs
- **Module 2: Program Synthesis in the era of Foundation Models**
 - Prompting strategies, evaluating coding LLMs
 - Iterative synthesis, MCP and tool use, agents, harness
- **Module 3: Applications of Program Synthesis**
 - Software engineering, security, theorem proving, planning, graphics
 - Robotics, HCI, AI4Science, high-performance computing
 - Other advanced topics

Dimensions in Program Synthesis



Today



Inductive Synthesis

=

Programming by Example

=

Inductive Program Synthesis

=

Inductive Programming

=

Inductive Learning

{“123”→“1”, “abc”→“a”} ➔ ???

{“123”→“1”, “abc”→“a”} ➔ input[0]

{“123”→“1”, “abc”→“a”}
{[0] → 1, [5,1] → 2}



input[0]



???

{“123”→“1”, “abc”→“a”}

{[0] → 1, [5,1] → 2}



input[0]



len(input)

{“123”→“1”, “abc”→“a”}
{[0] → 1, [5,1] → 2}



input[0], input[0:1], ...



len(input) , min(input) + 1, ...

High-level Picture

- Learning **abstraction** / **generalization** from a set of observations

Program Synthesis

$\{[0] \rightarrow 1, [5, 1] \rightarrow 2\}$ $\rightarrow$ `len(input)`
 $\{“123” \rightarrow “1”, “abc” \rightarrow “a”\}$ $\rightarrow$ `input[0]`

Deep Learning



MIT/LCS/TR-76

LEARNING STRUCTURAL DESCRIPTIONS FROM EXAMPLES

Patrick H. Winston

September 1970

LEARNING STRUCTURAL DESCRIPTIONS FROM EXAMPLES

Patrick H. Winston

September 1970

Abstract

The research here described centers on how a machine can recognize concepts and learn concepts to be recognized. Explanations are found in computer programs that build and manipulate abstract descriptions of scenes such as those children construct from toy blocks. One program uses sample scenes to create models of simple configurations like the three-brick arch. Another uses the resulting models in making identifications. Throughout emphasis is given to the importance of using good descriptions when exploring how machines can come to perceive and understand the visual environment.

High-level Picture

- Learning **abstraction** / **generalization** from a set of observations

Program Synthesis

$\{[0] \rightarrow 1, [5, 1] \rightarrow 2\} \rightarrow \text{min(input)} + 1$
 $\{“123” \rightarrow “1”, “abc” \rightarrow “a”\} \rightarrow \text{chars(input)[0]}$

Deep Learning



Space of Programs

Program Synthesis

$\{[0] \rightarrow 1, [5, 1] \rightarrow 2\} \rightarrow \text{len(input)}$
 $\{"123" \rightarrow "1", "abc" \rightarrow "a"\} \rightarrow \text{input}[0]$

Deep Learning

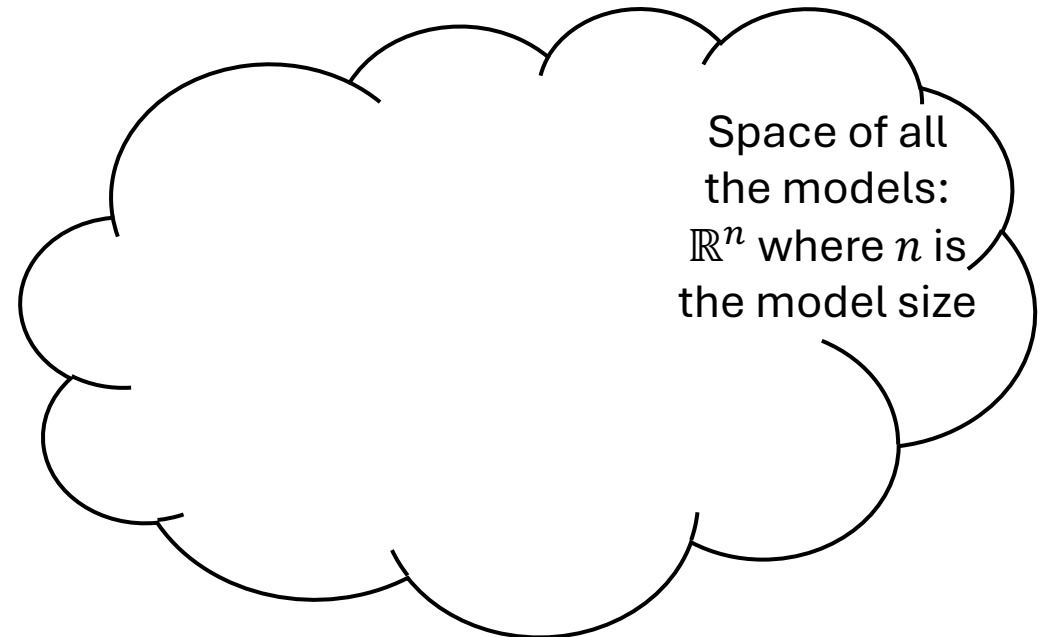


Space of Programs

Program Synthesis

$\{[0] \rightarrow 1, [5, 1] \rightarrow 2\}$ $\rightarrow$ `len(input)`
 $\{"123" \rightarrow "1", "abc" \rightarrow "a"\}$ $\rightarrow$ `input[0]`

Deep Learning

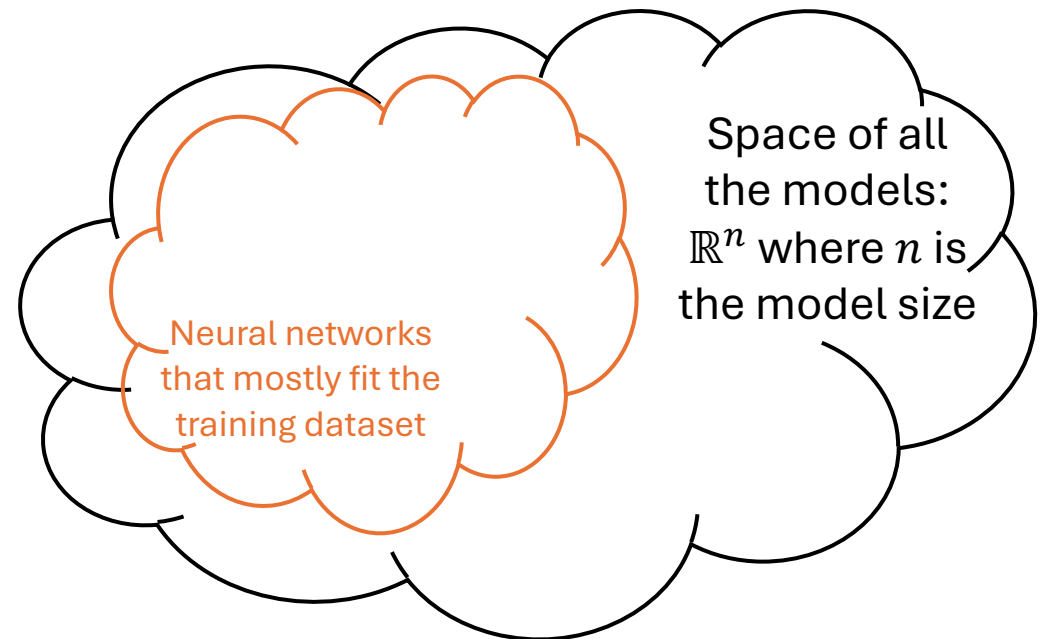


Space of Programs

Program Synthesis

$\{[0] \rightarrow 1, [5, 1] \rightarrow 2\}$ $\rightarrow$ `len(input)`
 $\{"123" \rightarrow "1", "abc" \rightarrow "a"\}$ $\rightarrow$ `input[0]`

Deep Learning

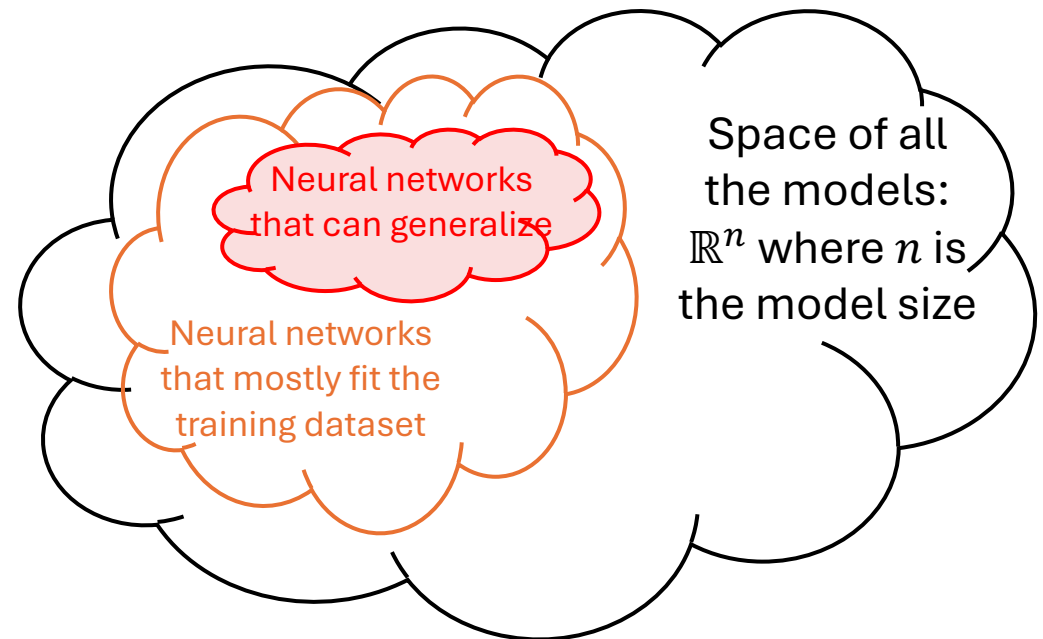


Space of Programs

Program Synthesis

$\{[0] \rightarrow 1, [5, 1] \rightarrow 2\} \rightarrow \text{len(input)}$
 $\{"123" \rightarrow "1", "abc" \rightarrow "a"\} \rightarrow \text{input}[0]$

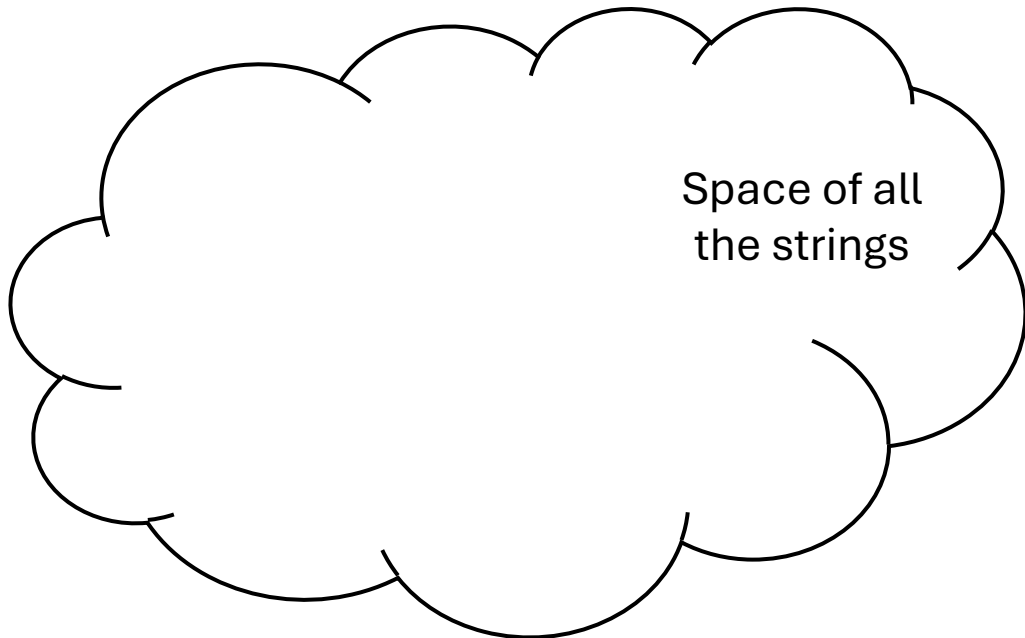
Deep Learning



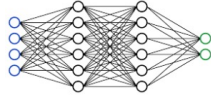
Space of Programs

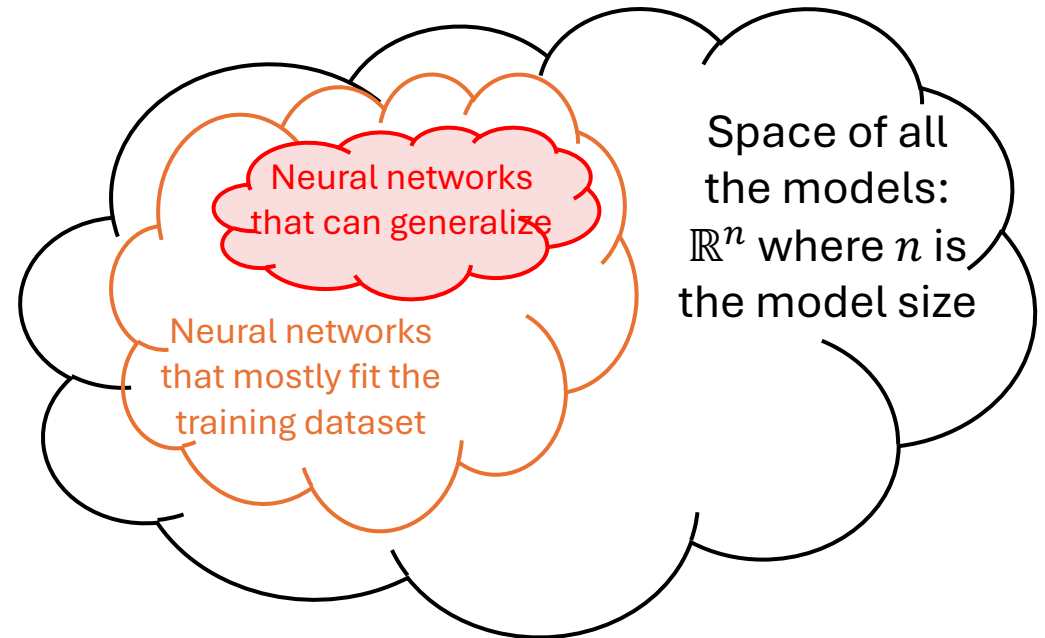
Program Synthesis

$\{[0] \rightarrow 1, [5, 1] \rightarrow 2\}$ $\rightarrow$ `len(input)`
 $\{"123" \rightarrow "1", "abc" \rightarrow "a"\}$ $\rightarrow$ `input[0]`



Deep Learning

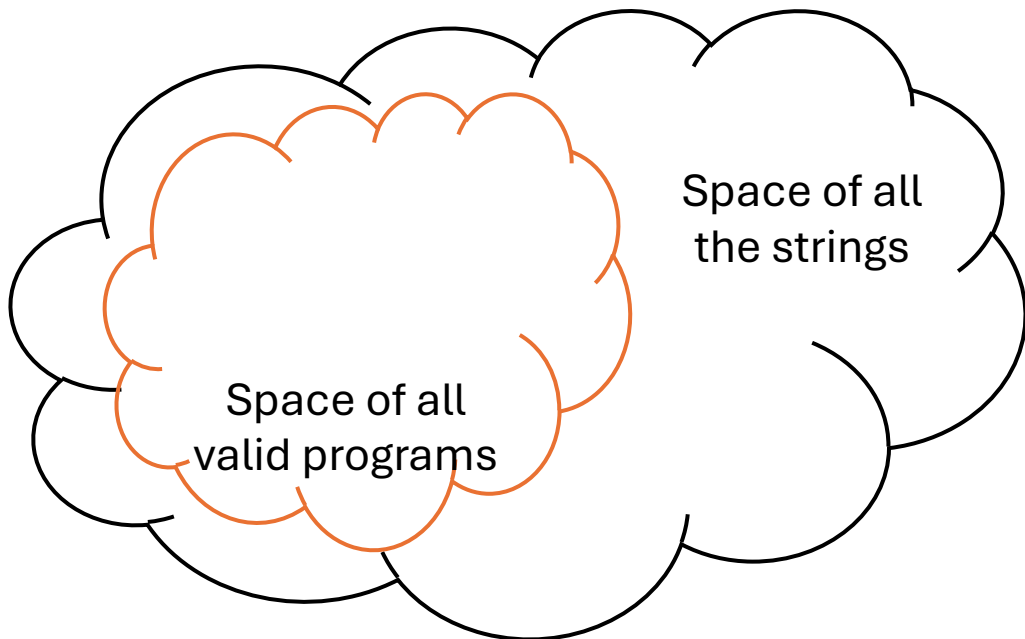
$\{ \text{dog image} \rightarrow \text{dog}, \text{cat image} \rightarrow \text{cat} \}$ $\rightarrow$ 



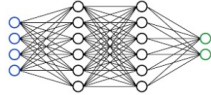
Space of Programs

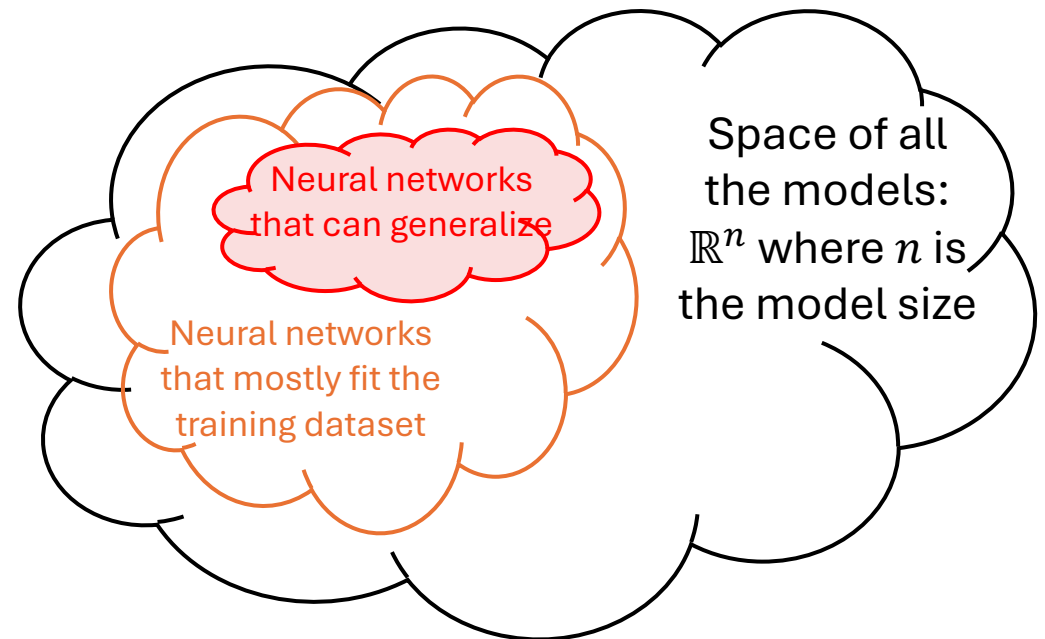
Program Synthesis

$\{[0] \rightarrow 1, [5, 1] \rightarrow 2\}$ $\rightarrow$ `len(input)`
 $\{"123" \rightarrow "1", "abc" \rightarrow "a"\}$ $\rightarrow$ `input[0]`



Deep Learning

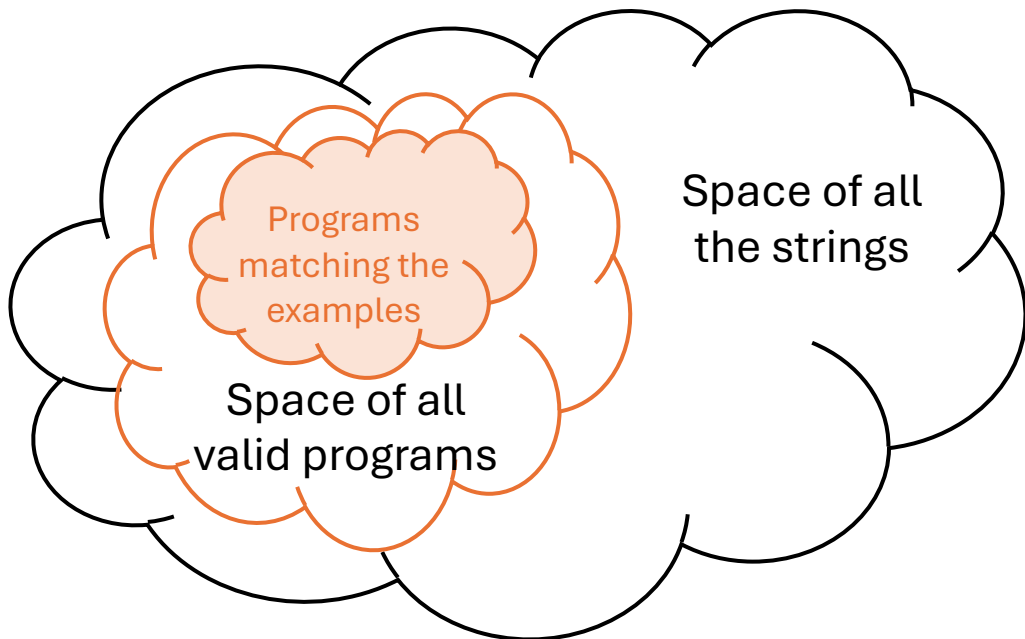
$\{ \text{dog image} \rightarrow \text{dog}, \text{cat image} \rightarrow \text{cat} \}$ $\rightarrow$ 



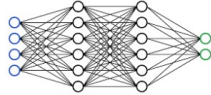
Space of Programs

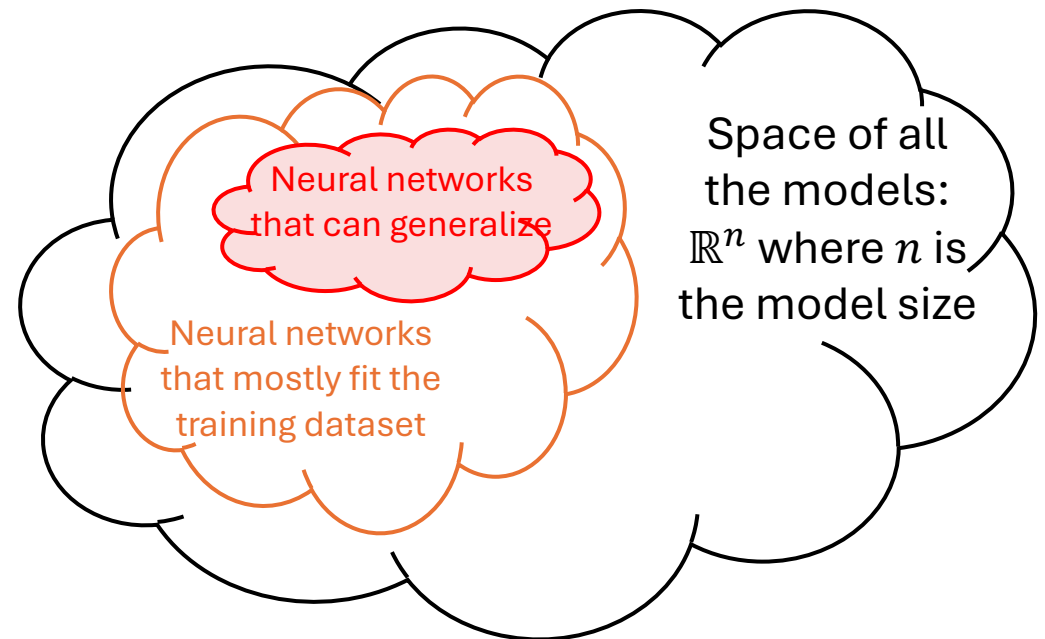
Program Synthesis

$\{[0] \rightarrow 1, [5, 1] \rightarrow 2\} \rightarrow \text{len(input)}$
 $\{"123" \rightarrow "1", "abc" \rightarrow "a"\} \rightarrow \text{input}[0]$



Deep Learning

$\{ \text{dog image} \rightarrow \text{dog}, \text{cat image} \rightarrow \text{cat} \} \rightarrow$ 

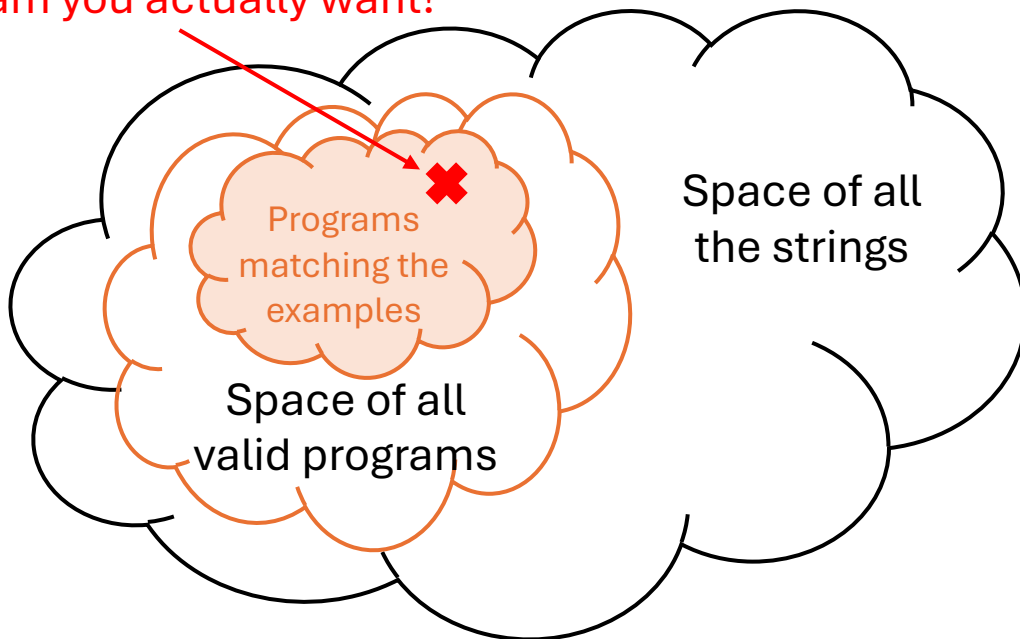


Space of Programs

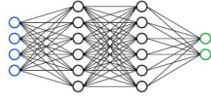
Program Synthesis

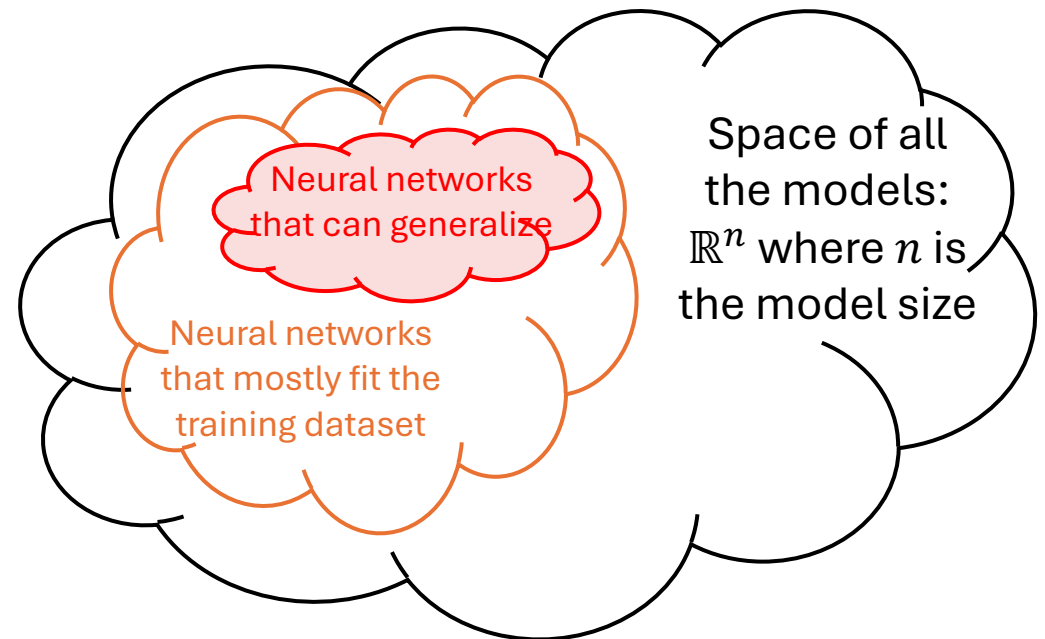
$\{[0] \rightarrow 1, [5, 1] \rightarrow 2\} \rightarrow \text{len(input)}$
 $\{"123" \rightarrow "1", "abc" \rightarrow "a"\} \rightarrow \text{input}[0]$

Program you actually want!



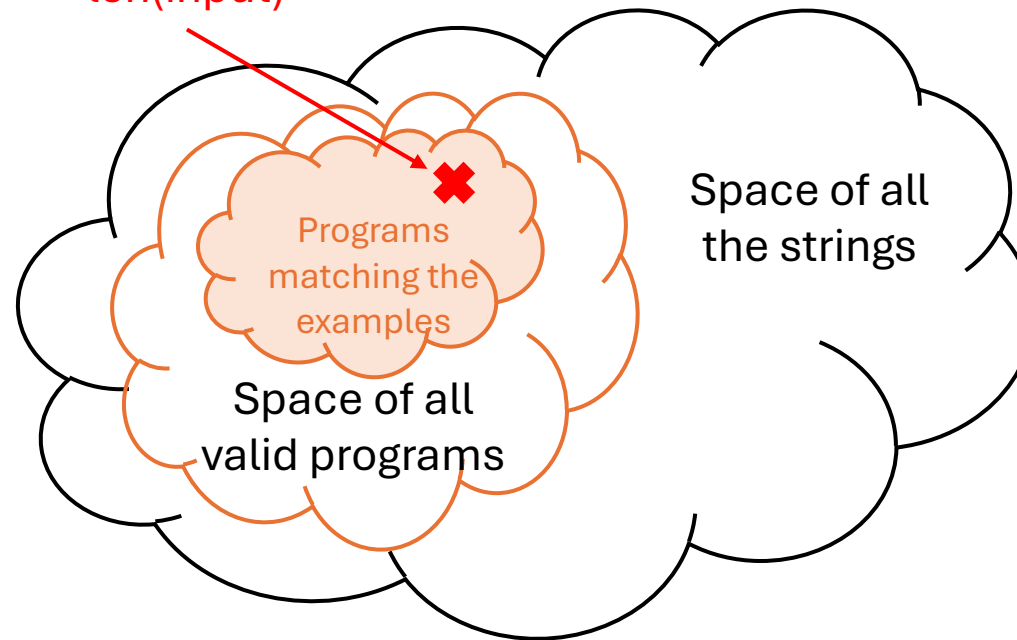
Deep Learning

$\{ \text{dog image} \rightarrow \text{dog}, \text{cat image} \rightarrow \text{cat} \} \rightarrow$ 

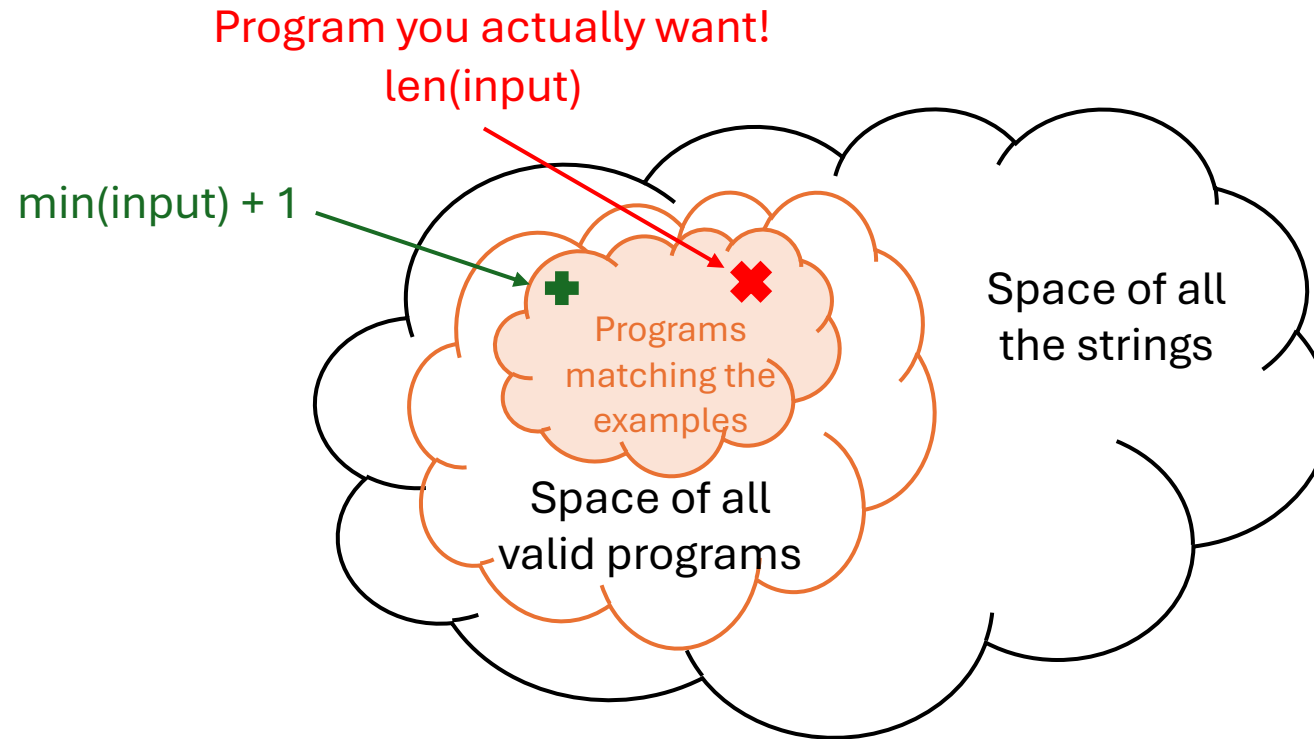


$\{[0] \rightarrow 1, [5,1] \rightarrow 2\} \rightarrow \text{len(input), min(input) + 1, ...}$

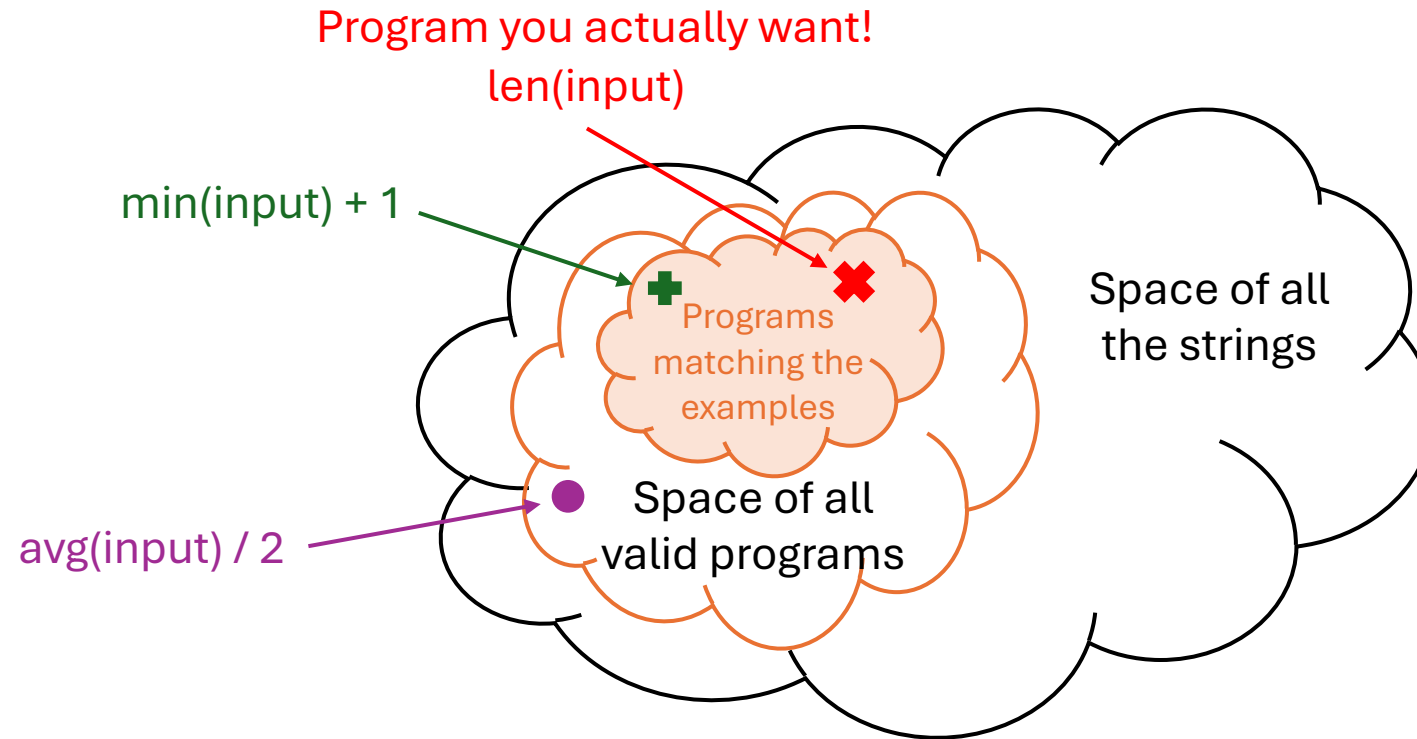
Program you actually want!
 len(input)



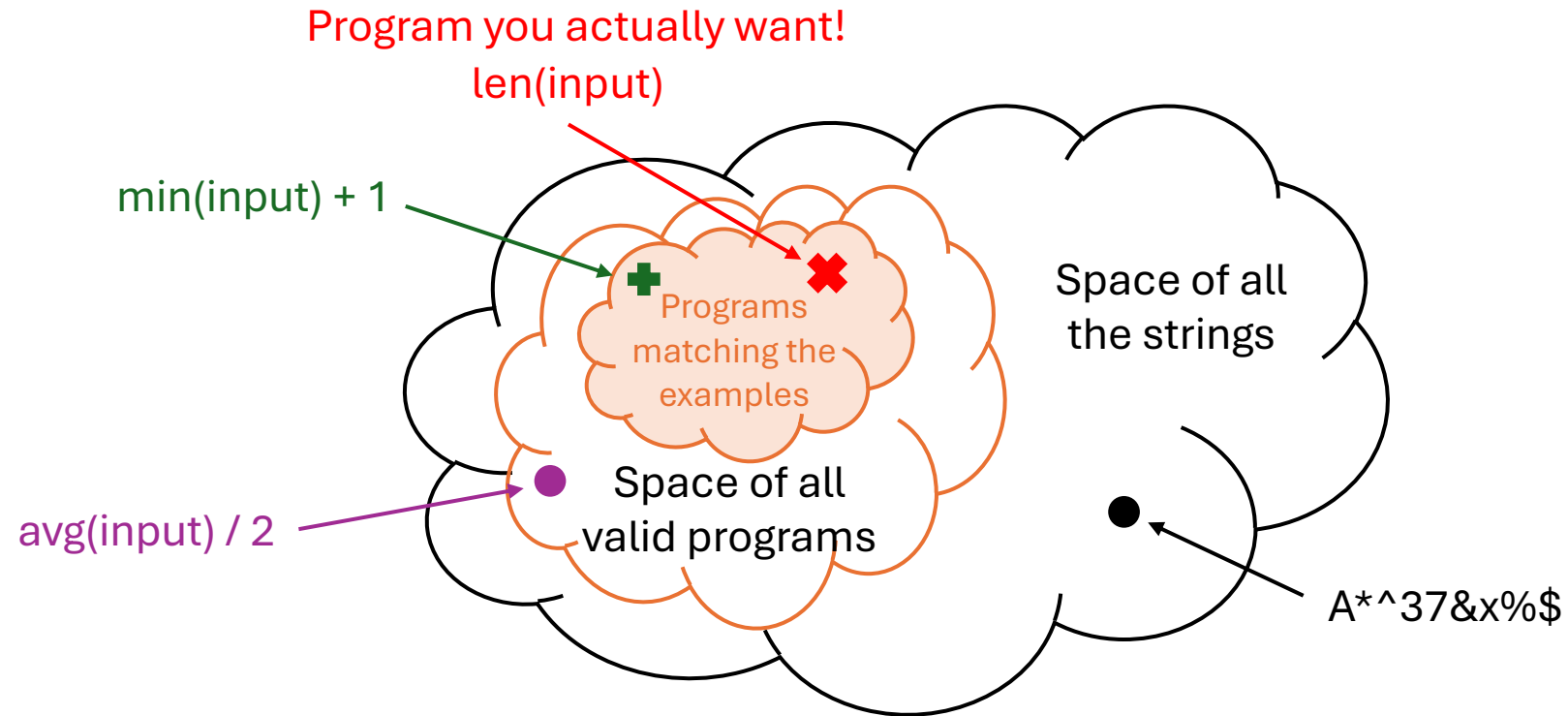
$\{[0] \rightarrow 1, [5,1] \rightarrow 2\} \rightarrow \text{len(input), min(input) + 1, ...}$



$\{[0] \rightarrow 1, [5,1] \rightarrow 2\} \rightarrow \text{len(input), min(input) + 1, ...}$



$\{[0] \rightarrow 1, [5,1] \rightarrow 2\} \rightarrow \text{len(input), min(input) + 1, ...}$

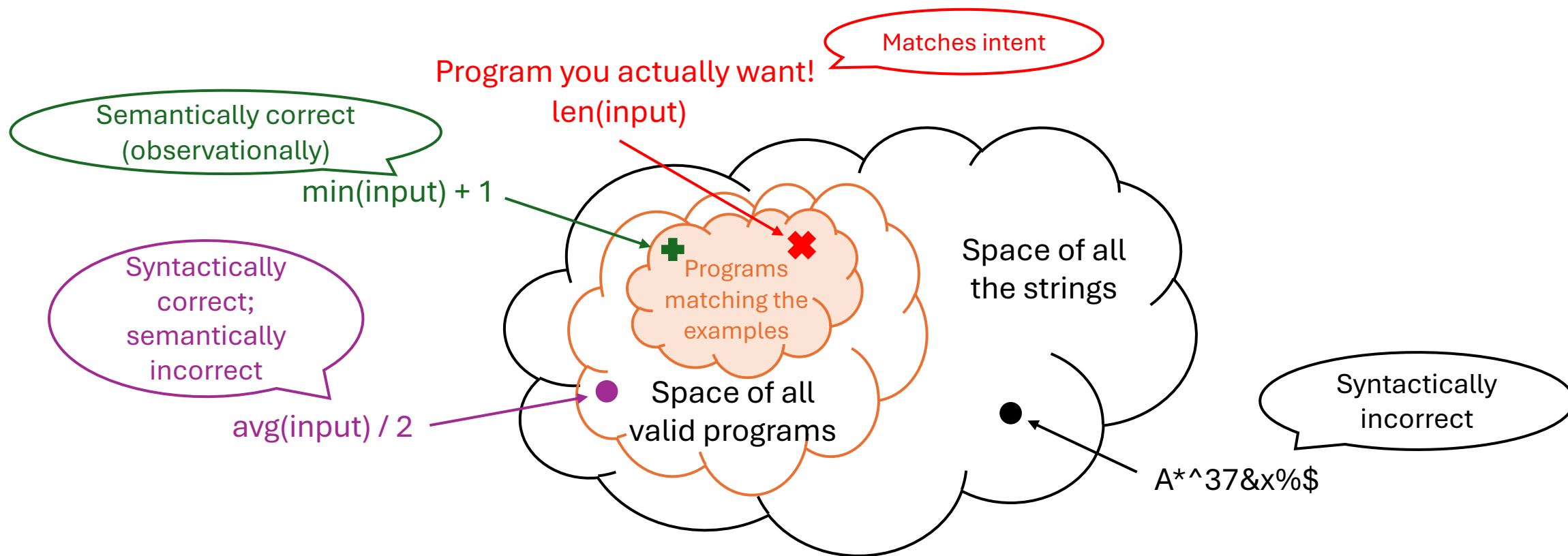


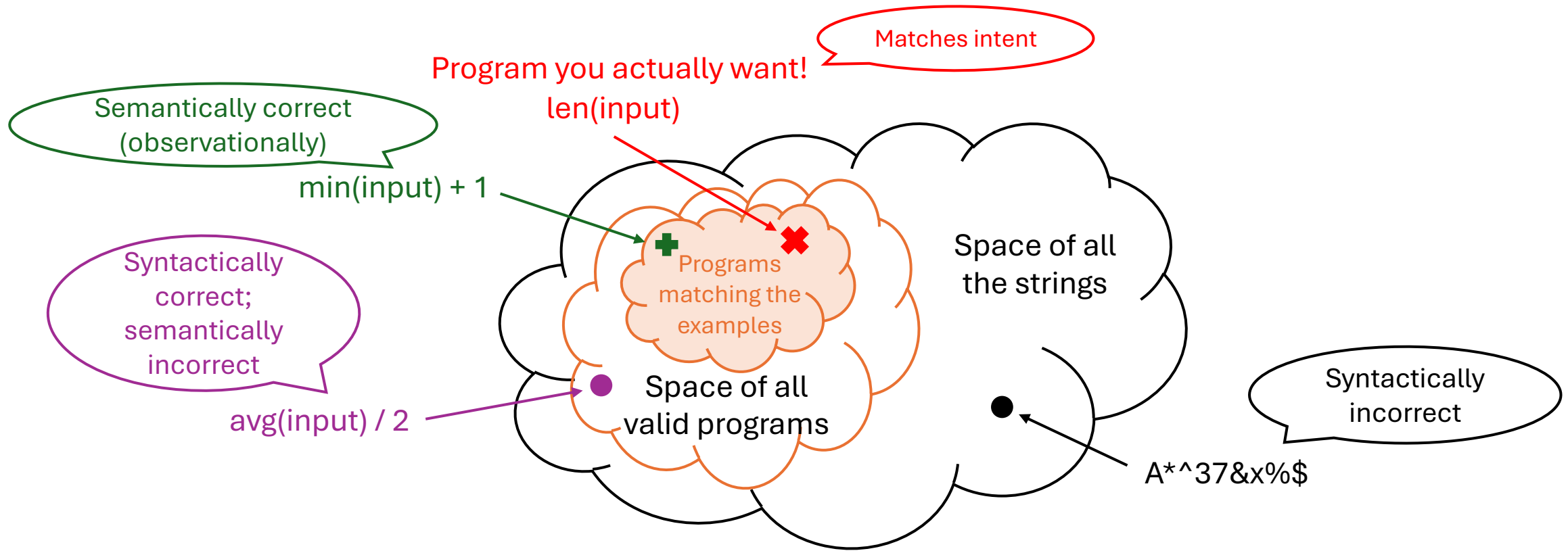
Behavioral Specification:

Examples

$\{[0] \rightarrow 1, [5,1] \rightarrow 2\}$

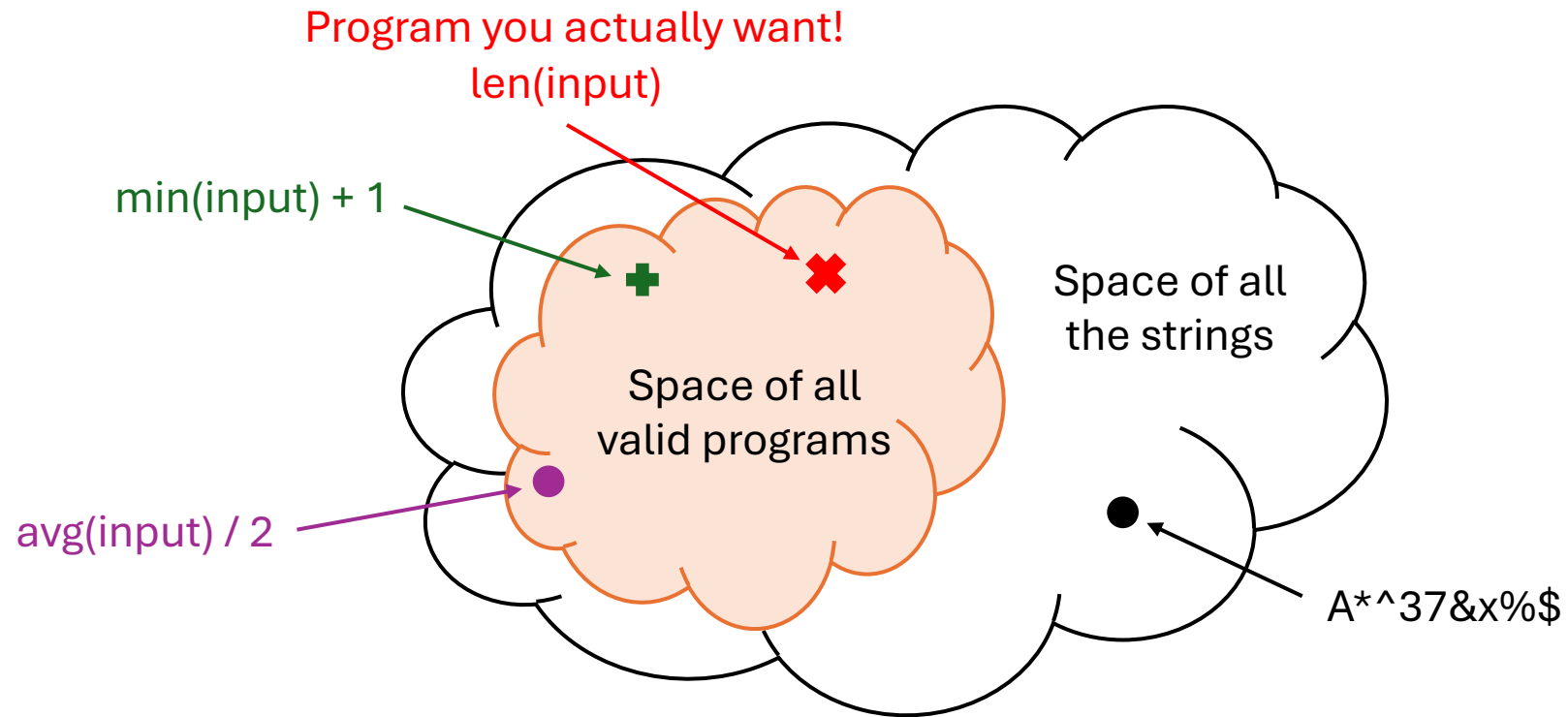
$\rightarrow \text{len(input)}, \text{min(input)} + 1, \dots$





Syntax + Semantics

Syntax



Syntax: Example

$\{[0] \rightarrow 1, [5,1] \rightarrow 2\}$

→ $\text{len}(\text{input}), \text{min}(\text{input}) + 1, \dots$

```
(Program) P ::= L | N
  (List) L ::= input
            | empty
            | single(N)
            | concat(L, L)
  (Number) N ::= len(L)
              | min(L)
              | add(N, N)
              | 0 | 1 | 2 | ...
```

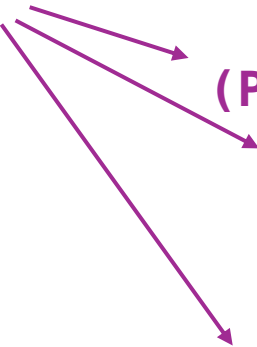
```
// either a list expr or number expr
// the input list
// []
// [N]
// concat([1],[2,3]) = [1,2,3]
// len([0]) = 1
// min([1,2]) = 1
// add(2,1) = 3
// the constant numerical literals
```

Syntax: Example

```
(Program) P ::= L | N
(List) L ::= input
          | empty
          | single(N)
          | concat(L, L)
(Number) N ::= len(L)
            | min(L)
            | add(N, N)
            | 0 | 1 | 2 | ...
```


Syntax: Backus-Naur Form (BNF)

Name of nonterminals

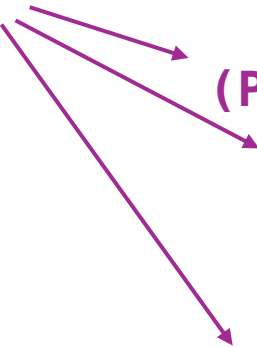


```
(Program) P ::= L | N
(List) L ::= input
          | empty
          | single(N)
          | concat(L, L)
(Number) N ::= len(L)
            | min(L)
            | add(N, N)
            | 0 | 1 | 2 | ...
```

Syntax: Backus-Naur Form (BNF)

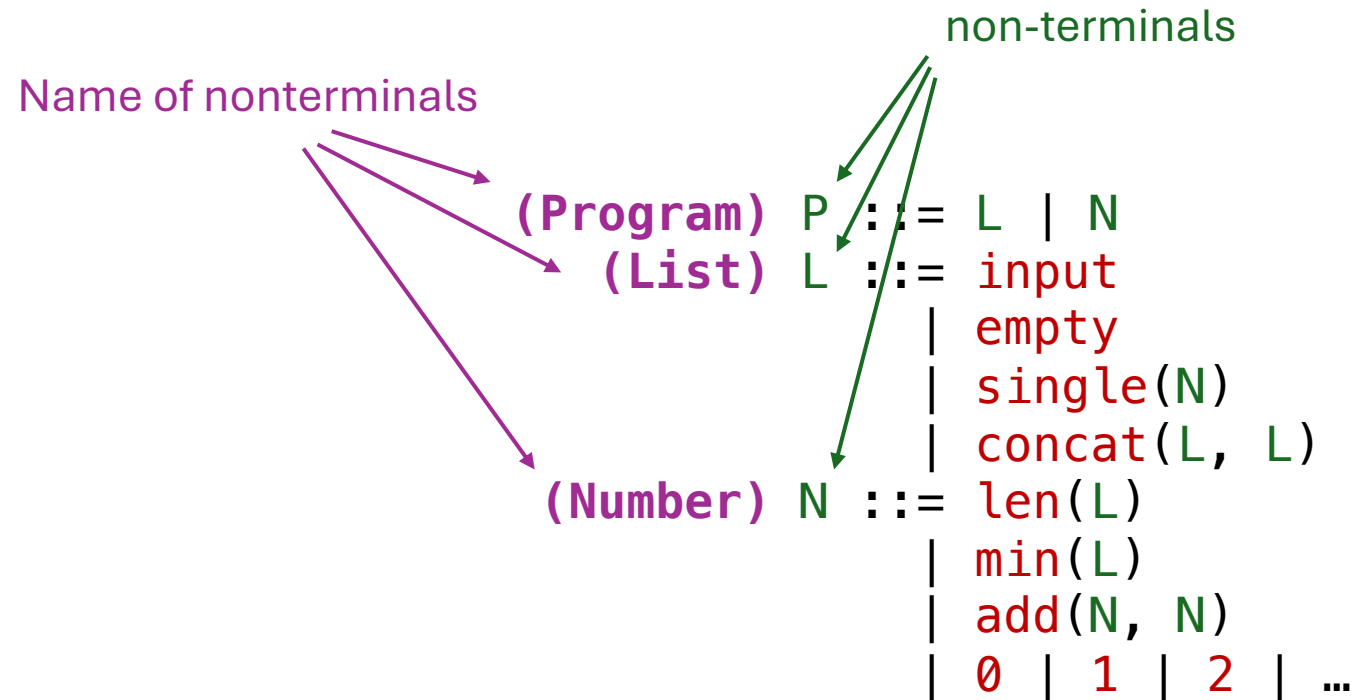
John Backus and Peter Naur created BNF in 1959–1960 to define the syntax of the ALGOL programming language. It acts as a metasyntax—a language used to describe another language. [Wikipedia +1](#)

Name of nonterminals

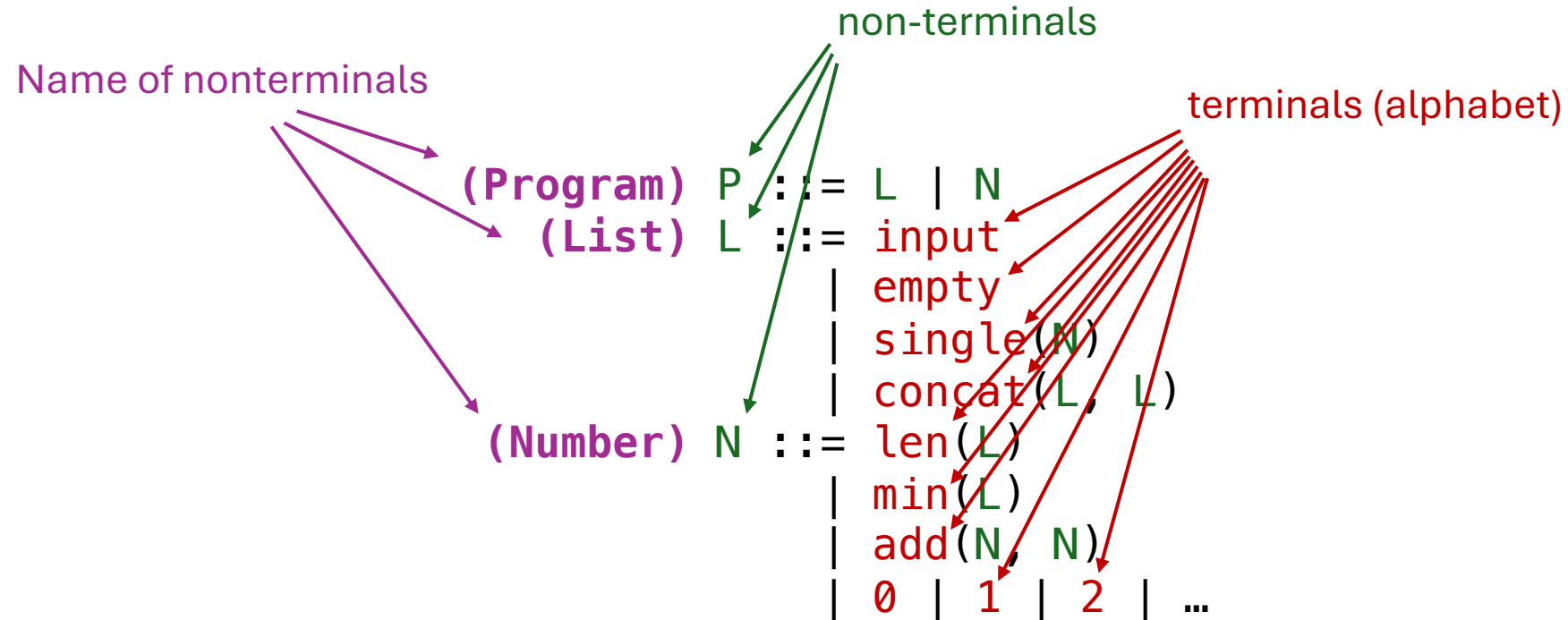


```
(Program) P ::= L | N
(List) L ::= input
          | empty
          | single(N)
          | concat(L, L)
(Number) N ::= len(L)
            | min(L)
            | add(N, N)
            | 0 | 1 | 2 | ...
```

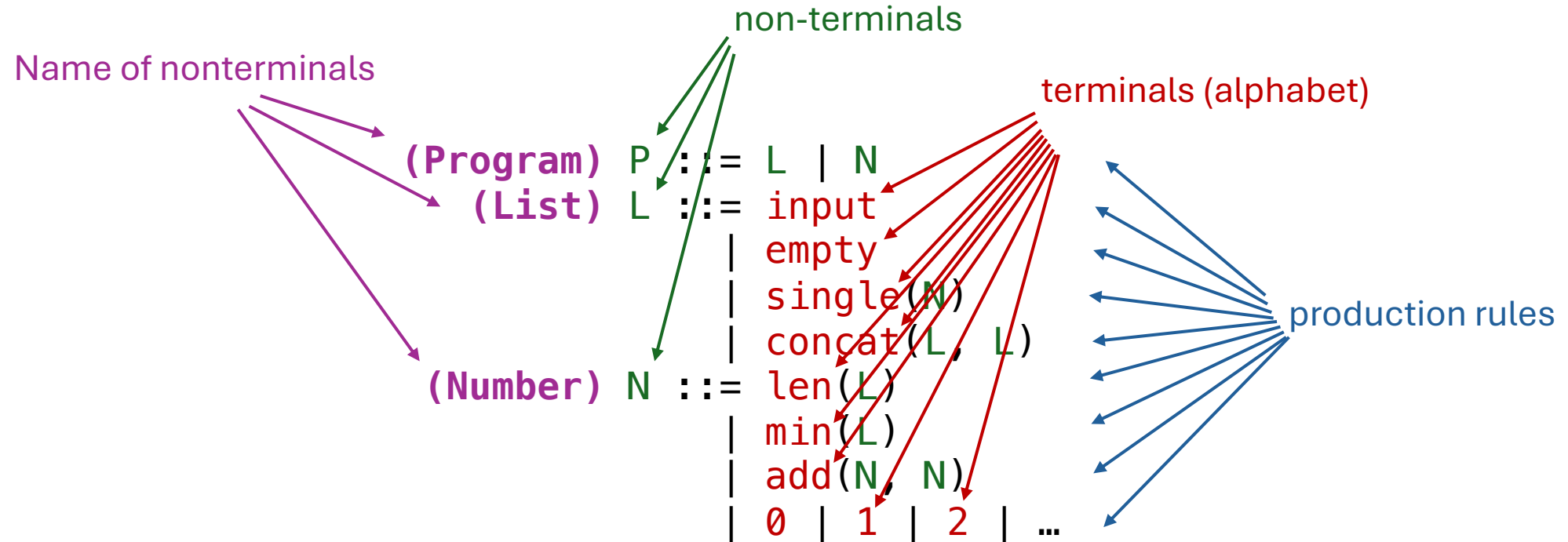
Syntax: Backus-Naur Form (BNF)



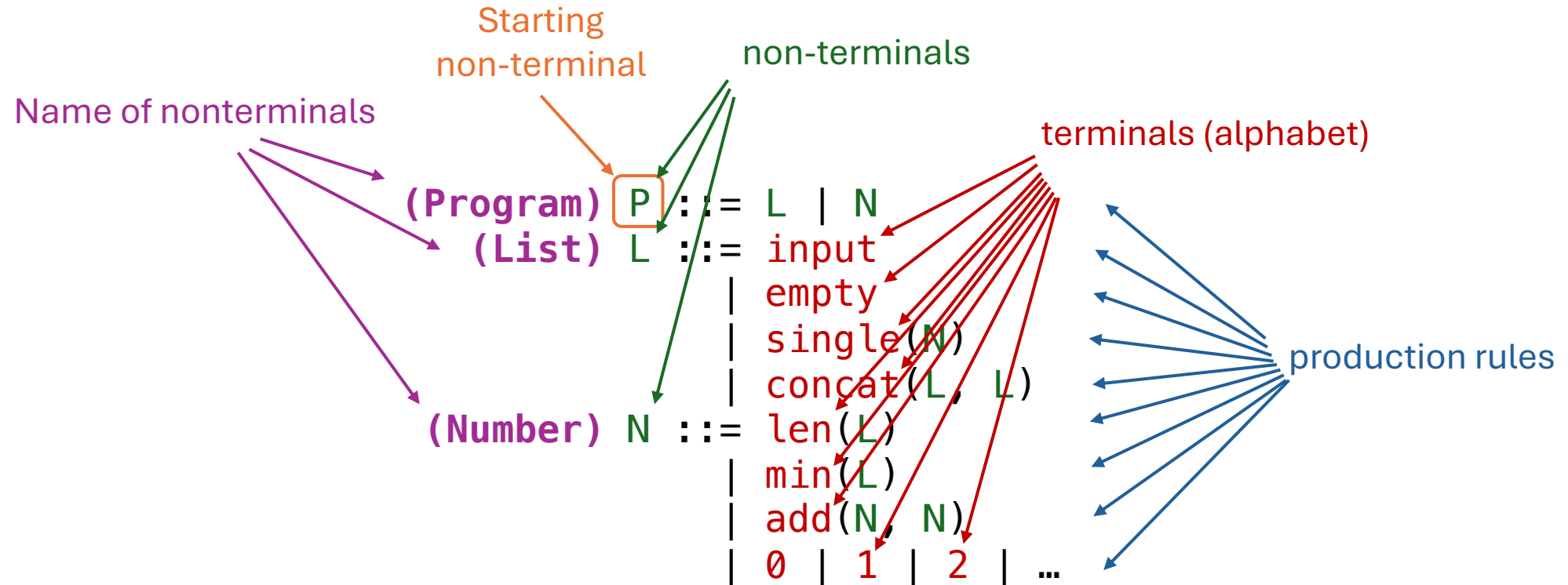
Syntax: Backus-Naur Form (BNF)



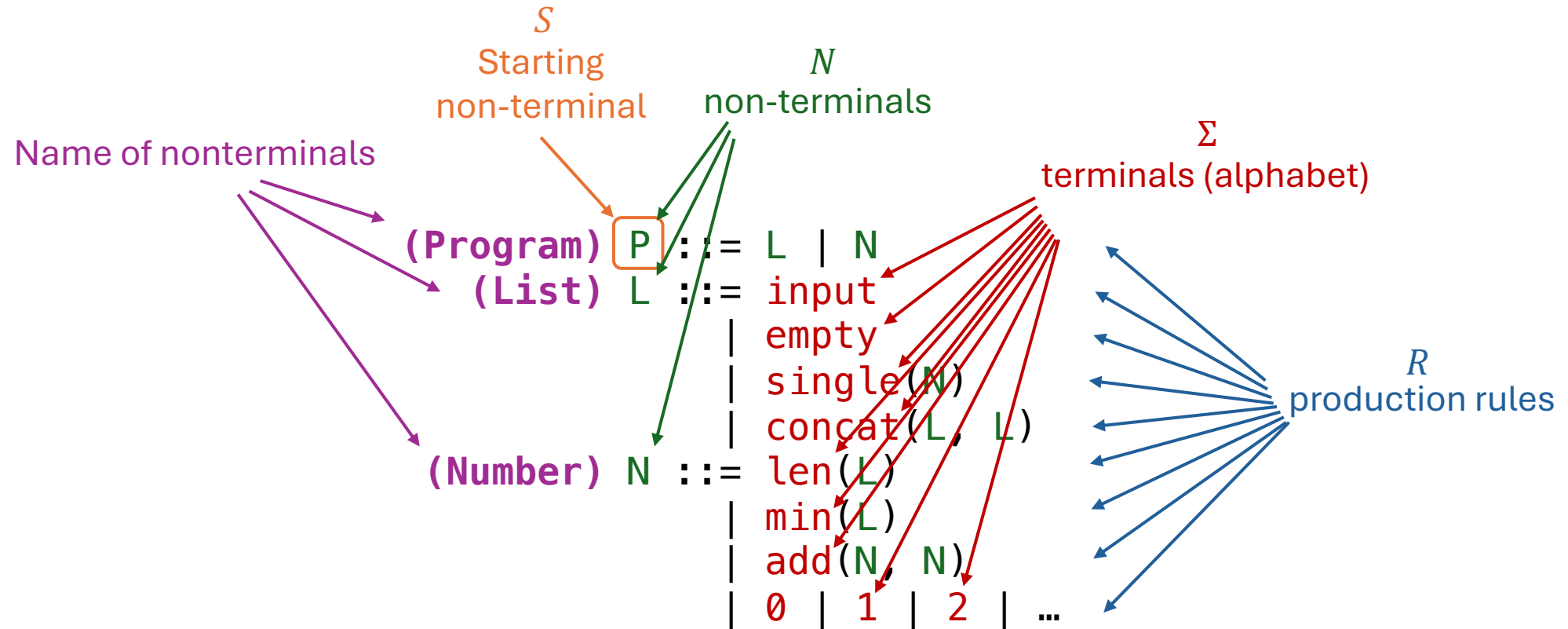
Syntax: Backus-Naur Form (BNF)



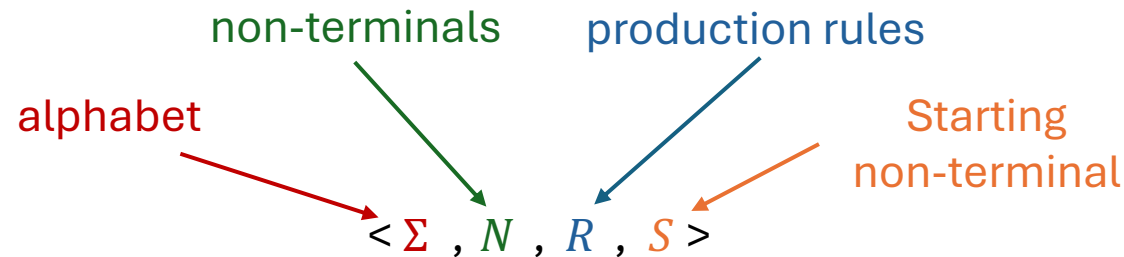
Syntax: Backus-Naur Form (BNF)



Syntax: Backus-Naur Form (BNF)



Syntax: Backus-Naur Form (BNF)



```
(Program) P ::= L | N
(List) L ::= input
           | empty
           | single(N)
           | concat(L, L)
(Number) N ::= len(L)
             | min(L)
             | add(N, N)
             | 0 | 1 | 2 | ...
```

Trees: $\tau \in T_{\Sigma}(N)$ = all trees made from $\Sigma \cup N$

Rules in R : $A \rightarrow \sigma(A_1, \dots, A_n)$ where $A \in N, A_i \in \Sigma \cup N$

Derivation in one step: $\rightarrow$

Derivations in multiple steps: $\rightarrow^*$

Incomplete Programs: a tree τ with non-terminals

- $\tau \in T_{\Sigma}(N)$ where $A \rightarrow^* \tau$

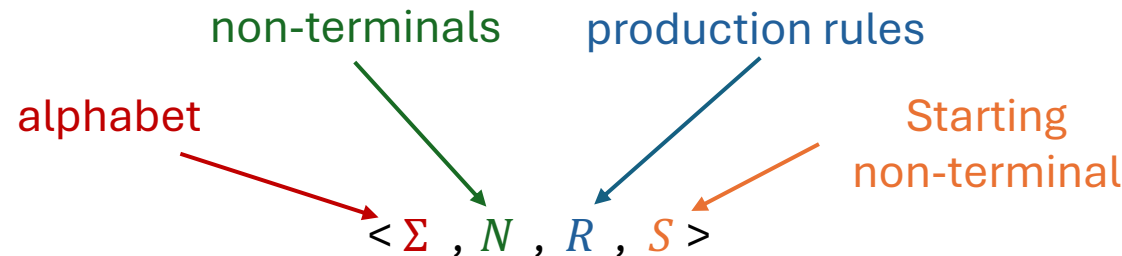
Complete Programs: a tree t without non-terminals

- $t \in T_{\Sigma}$ where $A \rightarrow^* t$

Whole Programs: a complete program t derivable by S

- $t \in T_{\Sigma}$ where $S \rightarrow^* t$

Syntax: Backus-Naur Form (BNF)



```

(Program) P ::= L | N
(List) L ::= input
           | empty
           | single(N)
           | concat(L, L)
(Number) N ::= len(L)
            | min(L)
            | add(N, N)
            | 0 | 1 | 2 | ...
    
```

Trees: $\tau \in T_{\Sigma}(N)$ = all trees made from $\Sigma \cup N$

Rules in R : $A \rightarrow \sigma(A_1, \dots, A_n)$ where $A \in N, A_i \in \Sigma \cup N$

Derivation in one step: $\rightarrow$

Derivations in multiple steps: $\rightarrow^*$

Incomplete Programs: a tree τ with non-terminals

- $\tau \in T_{\Sigma}(N)$ where $A \rightarrow^* \tau$

Complete Programs: a tree t without non-terminals

- $t \in T_{\Sigma}$ where $A \rightarrow^* t$

Whole Programs: a complete program t derivable by S

- $t \in T_{\Sigma}$ where $S \rightarrow^* t$

$\text{concat}(L, 0)$

$L \rightarrow \text{concat}(L, L)$

$\text{concat}(L, L) \rightarrow \text{concat}(\text{input}, L)$

$L \rightarrow^* \text{single}(\text{len}(L))$

$\text{len}(\text{concat}(L, L))$

$\text{len}(\text{concat}(\text{input}, \text{single}(1)))$

$\text{len}(\text{input})$

Syntax: Backus-Naur Form (BNF)

```
(Program)  $\underline{P}$  ::= L | N
(List) L ::= input
           | empty
           | single(N)
           | concat(L, L)
(Number) N ::= len(L)
            | min(L)
            | add(N, N)
            | 0 | 1 | 2 | ...
```

Space of programs

= the **language** of a BNF

= the language of an equivalent Regular Tree Grammar

= all **complete** & **whole** programs

What language is this?

```
if_stmt:
    | 'if' named_expression ':' block elif_stmt
    | 'if' named_expression ':' block [else_block]
elif_stmt:
    | 'elif' named_expression ':' block elif_stmt
    | 'elif' named_expression ':' block [else_block]
else_block:
    | 'else' ':' block

# While statement
# -----

while_stmt:
    | 'while' named_expression ':' block [else_block]

# For statement
# -----

for_stmt:
    | 'for' star_targets 'in' ~ star_expressions ':' [TYPE_COMMENT] block [else_block]
    | 'async' 'for' star_targets 'in' ~ star_expressions ':' [TYPE_COMMENT] block [el

# With statement
# -----

with_stmt:
    | 'with' '(' ','.with_item+ ','? ')' ':' [TYPE_COMMENT] block
    | 'with' ','.with_item+ ':' [TYPE_COMMENT] block
    | 'async' 'with' '(' ','.with_item+ ','? ')' ':' block
    | 'async' 'with' ','.with_item+ ':' [TYPE_COMMENT] block

with_item:
    | expression 'as' star_target &(',' | ')') | ':'
    | expression

# Try statement
# -----

try_stmt:
    | 'try' ':' block finally_block
    | 'try' ':' block except_block+ [else_block] [finally_block]
    | 'try' ':' block except_star_block+ [else_block] [finally_block]
```

Concrete and Abstract Syntax

Python Grammar (Concrete Syntax)

```
if_stmt:
    'if' named_expression ':' block elif_stmt
    'if' named_expression ':' block [else_block]
elif_stmt:
    'elif' named_expression ':' block elif_stmt
    'elif' named_expression ':' block [else_block]
else_block:
    'else' ':' block

# While statement
# -----
while_stmt:
    'while' named_expression ':' block [else_block]

# For statement
# -----
for_stmt:
    'for' star_targets 'in' ~ star_expressions ':' [TYPE_COMMENT] block [else_block]
    'async' 'for' star_targets 'in' ~ star_expressions ':' [TYPE_COMMENT] block [el

# With statement
# -----
with_stmt:
    'with' '(' ',' with_item+ ',' '?' ')' ':' [TYPE_COMMENT] block
    'with' ',' with_item+ ':' [TYPE_COMMENT] block
    'async' 'with' '(' ',' with_item+ ',' '?' ')' ':' block
    'async' 'with' ',' with_item+ ':' [TYPE_COMMENT] block

with_item:
    expression 'as' star_target &(',' | ')') | ':'

# Try statement
# -----
try_stmt:
    'try' ':' block finally_block
    'try' ':' block except_block+ [else_block] [finally_block]
    'try' ':' block except_star_block+ [else_block] [finally_block]
```

<https://docs.python.org/3/reference/grammar.html>

Python Abstract Syntax

```
-- use 'orelse' because else is a keyword in target languages
For(expr target, expr iter, stmt* body, stmt* orelse, string?
AsyncFor(expr target, expr iter, stmt* body, stmt* orelse, st
While(expr test, stmt* body, stmt* orelse)
If(expr test, stmt* body, stmt* orelse)
With(withitem* items, stmt* body, string? type_comment)
AsyncWith(withitem* items, stmt* body, string? type_comment)

Match(expr subject, match_case* cases)

Raise(expr? exc, expr? cause)
Try(stmt* body, excepthandler* handlers, stmt* orelse, stmt*
TryStar(stmt* body, excepthandler* handlers, stmt* orelse, st
Assert(expr test, expr? msg)

Import(alias* names)
ImportFrom(identifier? module, alias* names, int? level)

Global(identifier* names)
Nonlocal(identifier* names)
Expr(expr value)
Pass | Break | Continue

-- col_offset is the byte offset in the utf8 string the parser
attributes (int lineno, int col_offset, int? end_lineno, int? e

-- BoolOp() can use left & right?
expr = BoolOp(boolop op, expr* values)
NamedExpr(expr target, expr value)
BinOp(expr left, operator op, expr right)
UnaryOp(unaryop op, expr operand)
Lambda(arguments args, expr body)
IfExp(expr test, expr body, expr orelse)
Dict(expr* keys, expr* values)
Set(expr* elts)
ListComp(expr elt, comprehension* generators)
```

<https://docs.python.org/3/library/ast.html>

Syntax: How big is the space of programs?

$$E ::= x \mid f(E, E)$$

Syntax: How big is the space of programs?

$$E ::= x \mid f(E, E)$$

Depth ≤ 0



Size(0) = 1

Syntax: How big is the space of programs?

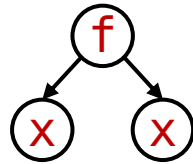
$E ::= x \mid f(E, E)$

Depth ≤ 0



$\text{Size}(0) = 1$

Depth ≤ 1



$\text{Size}(1) = 2$

Syntax: How big is the space of programs?

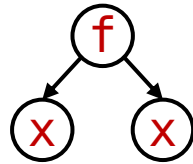
$E ::= x \mid f(E, E)$

Depth ≤ 0



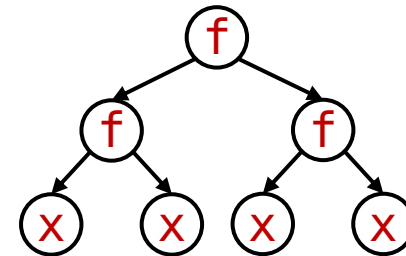
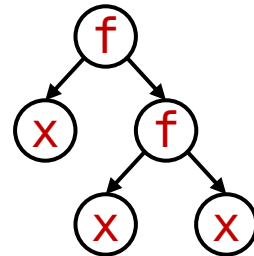
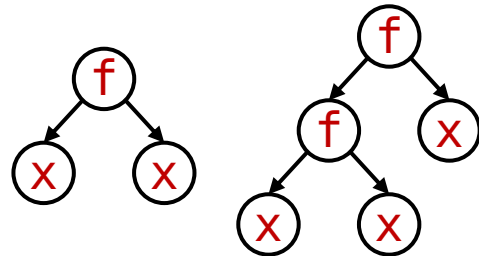
$\text{Size}(0) = 1$

Depth ≤ 1



$\text{Size}(1) = 2$

Depth ≤ 2



$\text{Size}(2) = 5$

Syntax: How big is the space of programs?

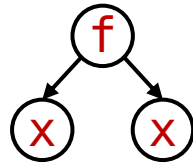
$E ::= x \mid f(E, E)$

Depth ≤ 0



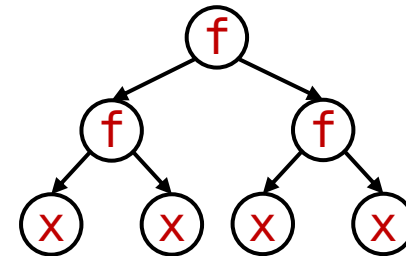
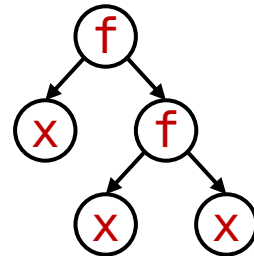
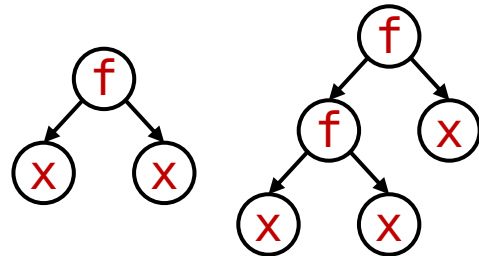
$\text{Size}(0) = 1$

Depth ≤ 1



$\text{Size}(1) = 2$

Depth ≤ 2



$\text{Size}(2) = 5$

$\text{Size}(\text{depth}) = ???$

Syntax: How big is the space of programs?

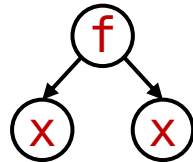
$E ::= x \mid f(E, E)$

Depth ≤ 0



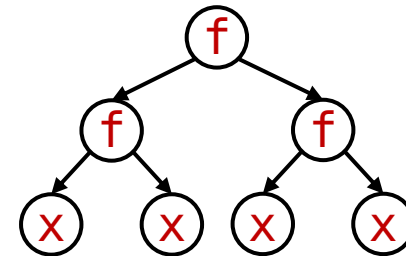
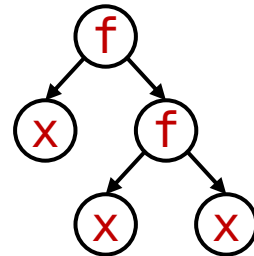
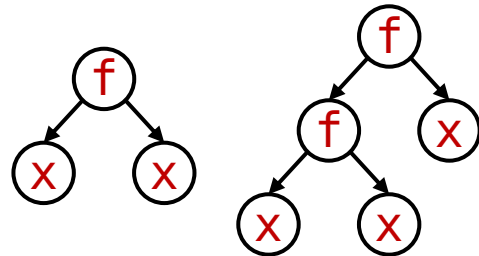
$\text{Size}(0) = 1$

Depth ≤ 1



$\text{Size}(1) = 2$

Depth ≤ 2



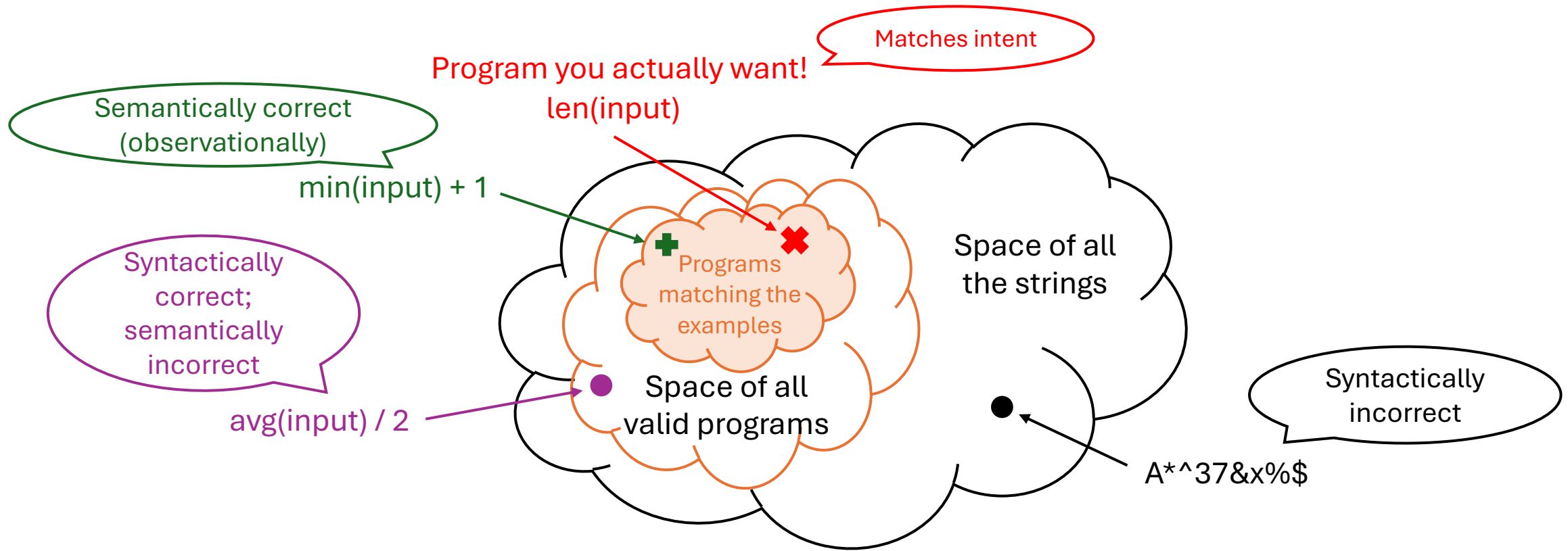
$\text{Size}(2) = 5$

$$\text{Size}(\text{depth}) = 1 + \text{Size}(\text{depth} - 1)^2$$

Syntax: How big is the space of programs?

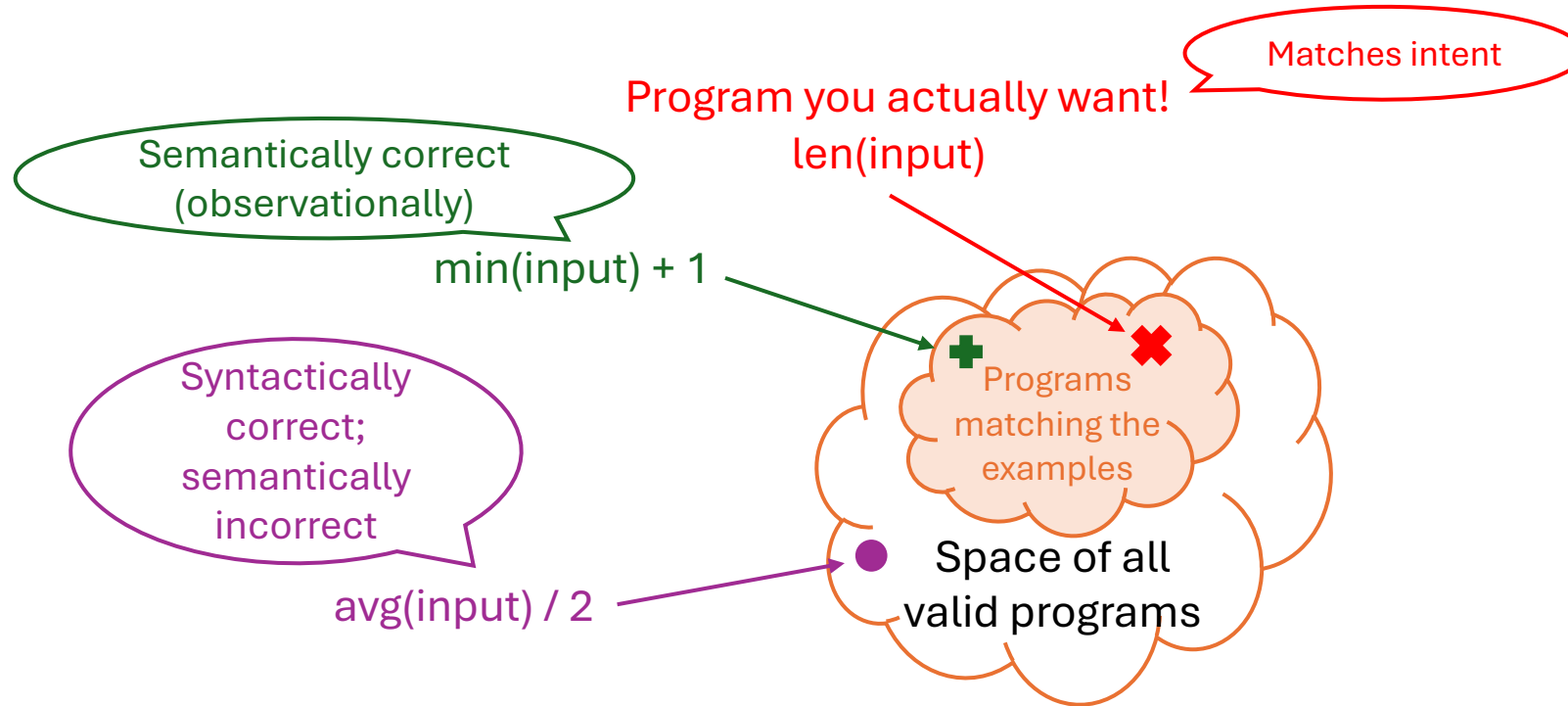
$$E ::= x \mid f(E, E)$$
$$\text{Size}(\text{depth}) = 1 + \text{Size}(\text{depth} - 1)^2$$

```
size(1) = 1
size(2) = 2
size(3) = 5
size(4) = 26
size(5) = 677
size(6) = 458330
size(7) = 210066388901
size(8) = 44127887745906175987802
size(9) = 1947270476915296449559703445493848930452791205
size(10) = 3791862310265926082868235028027893277370233152247388584761734150717768254410341175325352026
```



Syntax + Semantics

Semantics



Semantics: Meaning of a Language

(Program) $\underline{P} ::= L \mid N$
(List) $L ::= \text{input}$
 $\mid \text{empty}$
 $\mid \text{single}(N)$
 $\mid \text{concat}(L, L)$
(Number) $N ::= \text{len}(L)$
 $\mid \text{min}(L)$
 $\mid \text{add}(N, N)$
 $\mid 0 \mid 1 \mid 2 \mid \dots$

$eval : T_{\Sigma} \times IntList \rightarrow List \mid Int$
 $eval(\text{'input'}, x) = x$
 $eval(\text{'empty'}, x) = []$
 $\forall \tau \in T_{\Sigma}, eval(\text{'single'}(\tau), x) = [eval(\tau, x)]$
 $\forall \tau_1, \tau_2 \in T_{\Sigma}, eval(\text{'concat'}(\tau_1, \tau_2), x) = eval(\tau_1, x) + eval(\tau_2, x)$
 $\forall \tau \in T_{\Sigma}, eval(\text{'len'}(\tau), x) = |eval(\tau, x)|$
 $\forall \tau \in T_{\Sigma}, eval(\text{'min'}(\tau), x) = \min_v (v \in eval(\tau, x))$
 $\forall \tau_1, \tau_2 \in T_{\Sigma}, eval(\text{'add'}(\tau_1, \tau_2), x) = eval(\tau_1, x) + eval(\tau_2, x)$
 $eval(\text{'0'}, x) = 0, \dots$

Denotational Semantics

Mathematical meaning to each program construct

Semantics: Meaning of a Language

(Program) $\underline{P} ::= L \mid N$
(List) $L ::= \text{input}$
 $\mid \text{empty}$
 $\mid \text{single}(N)$
 $\mid \text{concat}(L, L)$
(Number) $N ::= \text{len}(L)$
 $\mid \text{min}(L)$
 $\mid \text{add}(N, N)$
 $\mid 0 \mid 1 \mid 2 \mid \dots$

$[[\cdot]] : T_{\Sigma} \times \text{IntList} \rightarrow \text{List} \mid \text{Int}$
 $[[\text{input}]](x) = x$
 $[[\text{empty}]](x) = []$
 $\forall \tau \in T_{\Sigma}, [[\text{single}(\tau)]](x) = [[[\tau]](x)]$
 $\forall \tau_1, \tau_2 \in T_{\Sigma}, [[\text{concat}(\tau_1, \tau_2)]](x) = [[\tau_1]](x) + [[\tau_2]](x)$
 $\forall \tau \in T_{\Sigma}, [[\text{len}(\tau)]](x) = | [[\tau]](x) |$
 $\forall \tau \in T_{\Sigma}, [[\text{min}(\tau)]](x) = \min_v (v \in [[\tau]](x))$
 $\forall \tau_1, \tau_2 \in T_{\Sigma}, [[\text{add}(\tau_1, \tau_2)]](x) = [[\tau_1]](x) + [[\tau_2]](x)$
 $[[0]](x) = 0, \dots$

Denotational Semantics

Mathematical meaning to each program construct

Semantics: Meaning of a Language

(Program) $\underline{P} ::= L \mid N$
(List) $L ::= \text{input} \mid \text{empty} \mid \text{single}(N) \mid \text{concat}(L, L)$
(Number) $N ::= \text{len}(L) \mid \text{min}(L) \mid \text{add}(N, N) \mid 0 \mid 1 \mid 2 \mid \dots$

$\frac{}{x \vdash \text{input} \Downarrow x}$	$\frac{}{x \vdash \text{empty} \Downarrow []}$
$\frac{x \vdash N \Downarrow n}{x \vdash \text{single}(N) \Downarrow [n]}$	$\frac{x \vdash L1 \Downarrow vL1 \quad x \vdash L2 \Downarrow vL2}{x \vdash \text{concat}(L1, L2) \Downarrow vL1 ++ vL2}$
...	

Operational Semantics

Meaning in terms of computation steps

Semantics: Meaning of a Language

```
(Program)  $\underline{P} ::= L \mid N$   
  (List)  $L ::= \text{input}$   
           $\mid \text{empty}$   
           $\mid \text{single}(N)$   
           $\mid \text{concat}(L, L)$   
  (Number)  $N ::= \text{len}(L)$   
             $\mid \text{min}(L)$   
             $\mid \text{add}(N, N)$   
             $\mid 0 \mid 1 \mid 2 \mid \dots$ 
```

```
def evaluate(program, input):  
    if isinstance(program, Empty):  
        return []  
    elif isinstance(program, Input):  
        return input  
    elif isinstance(program, Concat):  
        left = evaluate(program.left, input)  
        right = evaluate(program.right, input)  
        return left + right  
    elif ...
```

Semantics written in Python...

Meaning encoded as an evaluator / interpreter

Datafun

$\llbracket A \rrbracket \in \text{Poset}_0$
 $\llbracket 2 \rrbracket = 2$
 $\llbracket N \rrbracket = N_{\leq}$
 $\llbracket \text{str} \rrbracket = \text{Disc } \mathbb{S}$
 $\llbracket A \times B \rrbracket = \llbracket A \rrbracket \times \llbracket B \rrbracket$
 $\llbracket A + B \rrbracket = \llbracket A \rrbracket + \llbracket B \rrbracket$
 $\llbracket A \xrightarrow{+} B \rrbracket = \llbracket A \rrbracket \Rightarrow \llbracket B \rrbracket$
 $\llbracket A \rightarrow B \rrbracket = \text{Disc } |\llbracket A \rrbracket| \Rightarrow \llbracket B \rrbracket$
 $\llbracket \{A\} \rrbracket = \mathcal{P}_{\text{fin}} |\llbracket A \rrbracket|$

$\llbracket \Delta \rrbracket, \llbracket \Gamma \rrbracket \in \text{Poset}_0$
 $\llbracket \cdot \rrbracket = 1$
 $\llbracket \Delta, x:A \rrbracket = \llbracket \Delta \rrbracket \times \llbracket A \rrbracket$
 $\llbracket \Gamma, x:A \rrbracket = \llbracket \Gamma \rrbracket \times \llbracket A \rrbracket$

Signature

$\text{alloc}(\tau_1)$
 $\overline{d}_m \leftarrow \text{ev}$
 $\overline{d}_n \leftarrow \text{gather}(i, s_n)$
 $d \leftarrow \text{gather}(\alpha_{n,1})(\overline{i}_n, \overline{s}_n)$
 $\text{store}(\rho)(\overline{s}_n, s_r)$
 $[\overline{s}_n, s_r] = \text{load}(\rho)()$
 $\overline{d}_n \leftarrow \text{build}(\overline{s}_n)$
 $\overline{d}_n \leftarrow \text{count}(\overline{b}_n, h, \overline{a}_n)$
 $\overline{d}_n \leftarrow \text{scan}(s)$
 $[d_i, d_r] \leftarrow \text{join}(W)(\overline{b}_m, \overline{a}_n, h, c, o)$
 $\overline{d}_n \leftarrow \text{copy}(\overline{s}_n)$
 $\overline{d}_n \leftarrow \text{sort}(\overline{s}_n)$
 $[\overline{d}_n, s] \leftarrow \text{unique}(\sigma)(\overline{s}_n)$
 $\overline{d}_n \leftarrow \text{merge}(\overline{a}_n, \overline{b}_n)$

Gather rows of s_n
 Gather rows of $\overline{s}_n$
 Store registers $\overline{s}_n$
 Loads the columns and tags of relation ρ with arity n from the data
 Builds a hash index for register the table with columns $\overline{s}_n$.
 Count the number of occurrences of each tuple columns $\overline{a}_n$ via the hash index h .
 Computes the (exclusive) prefix sum of registers
 Produces the resulting indices from a W column the hash index h and count c and offset o .
 Copies from register $\overline{s}_n$, truncating if the destination
 Lexicographically sorts the table with columns
 Merges adjacent duplicate rows via σ from the unique elements s .
 Merges two sorted tables with columns $\overline{a}_n$ and

Lobster

SQL

WebAssembly (WASM)

Reduction

$$\frac{s; v^*; e^* \hookrightarrow_i s'; v'^*; e'^*}{s; v^*; L^k[e^*] \hookrightarrow_i s'; v'^*; L^k[e'^*]} \quad \frac{s; v^*; e^* \hookrightarrow_i s'; v'^*; e'^*}{s; v_0^*; \text{local}_n\{i; v^*\} e^* \text{ end} \hookrightarrow_j s'; v_0^*; \text{local}_n\{i; v'^*\} e'^* \text{ end}}$$

$$L^0[\text{trap}] \hookrightarrow \text{trap} \quad \text{if } L^0 \neq [-]$$

$$\begin{aligned} (t.\text{const } c) t.\text{unop} &\hookrightarrow t.\text{const } \text{unop}_t(c) \\ (t.\text{const } c_1) (t.\text{const } c_2) t.\text{binop} &\hookrightarrow t.\text{const } c \quad \text{if } c = \text{binop}_t(c_1, c_2) \\ (t.\text{const } c_1) (t.\text{const } c_2) t.\text{binop} &\hookrightarrow \text{trap} \quad \text{otherwise} \\ (t.\text{const } c) t.\text{testop} &\hookrightarrow \text{i32.const } \text{testop}_t(c) \\ (t.\text{const } c_1) (t.\text{const } c_2) t.\text{relop} &\hookrightarrow \text{i32.const } \text{relop}_t(c_1, c_2) \\ (t_1.\text{const } c) t_2.\text{convert } t_1\text{-sx}^? &\hookrightarrow t_2.\text{const } c' \quad \text{if } c' = \text{cvt}_{t_1, t_2}^{\text{sx}}(c) \\ (t_1.\text{const } c) t_2.\text{convert } t_1\text{-sx}^? &\hookrightarrow \text{trap} \quad \text{otherwise} \\ (t_1.\text{const } c) t_2.\text{reinterpret } t_1 &\hookrightarrow t_2.\text{const } \text{const}_{t_2}(\text{bits}_{t_1}(c)) \\ \text{unreachable} &\hookrightarrow \text{trap} \end{aligned}$$

Expression semantics

$\alpha : \mathcal{U} \rightarrow \mathcal{U}, \quad \beta : \mathcal{U} \rightarrow \text{Bool}, \quad g : \mathcal{U} \rightarrow \mathcal{U}, \quad \llbracket e \rrbracket : \mathcal{F}_T \rightarrow \mathcal{U}_T$

$$\frac{t :: p(u) \in F_T}{t :: u \in \llbracket p \rrbracket(F_T)} \text{ (PREDICATE)} \quad \frac{t :: u \in \llbracket e \rrbracket(F_T) \quad \beta(u) = \text{true}}{t :: u \in \llbracket \sigma_\beta(e) \rrbracket(F_T)} \text{ (SELECT)} \quad \frac{t :: u \in \llbracket e \rrbracket(F_T) \quad u' = \alpha(u)}{t :: u' \in \llbracket \pi_\alpha(e) \rrbracket(F_T)} \text{ (PROJECT)}$$

$$\frac{t :: u_1 \in \llbracket e_1 \rrbracket(F_T) \quad t :: u_2 \in \llbracket e_2 \rrbracket(F_T)}{t :: u_1 \times u_2 \in \llbracket e_1 \times e_2 \rrbracket(F_T)} \text{ (PRODUCT)}$$

$$\frac{t_2 :: u \in \llbracket e_2 \rrbracket(F_T)}{t_1 :: u \in \llbracket e_1 - e_2 \rrbracket(F_T)} \text{ (DIFF-2)}$$

$$\frac{u \in g(\{u_i\}_{i=1}^n)}{t :: u \in \llbracket g \rrbracket(F_T)} \text{ (AGGREGATE)}$$

$$\llbracket R \rrbracket_{D, \eta, x} = R^D$$

$$\llbracket \tau : \beta \rrbracket_{D, \eta, x} = \llbracket T_1 \rrbracket_{D, \eta, 0} \times \cdots \times \llbracket T_k \rrbracket_{D, \eta, 0} \quad \text{for } \tau = (T_1, \dots, T_k)$$

$$\left[\begin{array}{l} \text{FROM } \tau : \beta \\ \text{WHERE } \theta \end{array} \right]_{D, \eta, x} = \left\{ \left. \begin{array}{l} \bar{r}, \dots, \bar{r} \\ k \text{ times} \end{array} \right| \bar{r} \in k \llbracket \tau : \beta \rrbracket_{D, \eta, 0}, \llbracket \theta \rrbracket_{D, \eta'} = \text{t}, \eta' = \eta \oplus \ell(\tau : \beta) \right\}$$

$$\left[\begin{array}{l} \text{SELECT } \alpha : \beta' \\ \text{FROM } \tau : \beta \\ \text{WHERE } \theta \end{array} \right]_{D, \eta, x} = \left\{ \left. \begin{array}{l} \llbracket \alpha \rrbracket_{\eta'}, \dots, \llbracket \alpha \rrbracket_{\eta'} \\ k \text{ times} \end{array} \right| \eta' = \eta \oplus \ell(\tau : \beta), \bar{r} \in k \left[\begin{array}{l} \text{FROM } \tau : \beta \\ \text{WHERE } \theta \end{array} \right]_{D, \eta, x} \right\}$$

$$\left[\begin{array}{l} \text{SELECT } * \\ \text{FROM } \tau : \beta \\ \text{WHERE } \theta \end{array} \right]_{D, \eta, 0} = \left[\begin{array}{l} \text{SELECT } \ell(\tau : \beta) : \ell(\tau) \\ \text{FROM } \tau : \beta \\ \text{WHERE } \theta \end{array} \right]_{D, \eta, 0}$$

$$\left[\begin{array}{l} \text{SELECT } * \\ \text{FROM } \tau : \beta \\ \text{WHERE } \theta \end{array} \right]_{D, \eta, 1} = \left[\begin{array}{l} \text{SELECT } c \text{ AS } N \\ \text{FROM } \tau : \beta \\ \text{WHERE } \theta \end{array} \right]_{D, \eta, 1} \quad \text{for arbitrary } c \in \mathbb{C} \text{ and } N \in \mathbb{N}$$

$$\left[\begin{array}{l} \text{SELECT DISTINCT } \alpha : \beta' \mid * \\ \text{FROM } \tau : \beta \text{ WHERE } \theta \end{array} \right]_{D, \eta, x} = \varepsilon \left(\left[\begin{array}{l} \text{SELECT } \alpha : \beta' \mid * \\ \text{FROM } \tau : \beta \text{ WHERE } \theta \end{array} \right]_{D, \eta, x} \right)$$

Scallop

Grounded with concrete inputs...

Syntax

```
(Program)  $\underline{P} ::= L \mid N$   
(List)  $L ::= \text{input}$   
          |  $\text{empty}$   
          |  $\text{single}(N)$   
          |  $\text{concat}(L, L)$   
(Number)  $N ::= \text{len}(L)$   
           |  $\text{min}(L)$   
           |  $\text{add}(N, N)$   
           |  $0 \mid 1 \mid 2 \mid \dots$ 
```

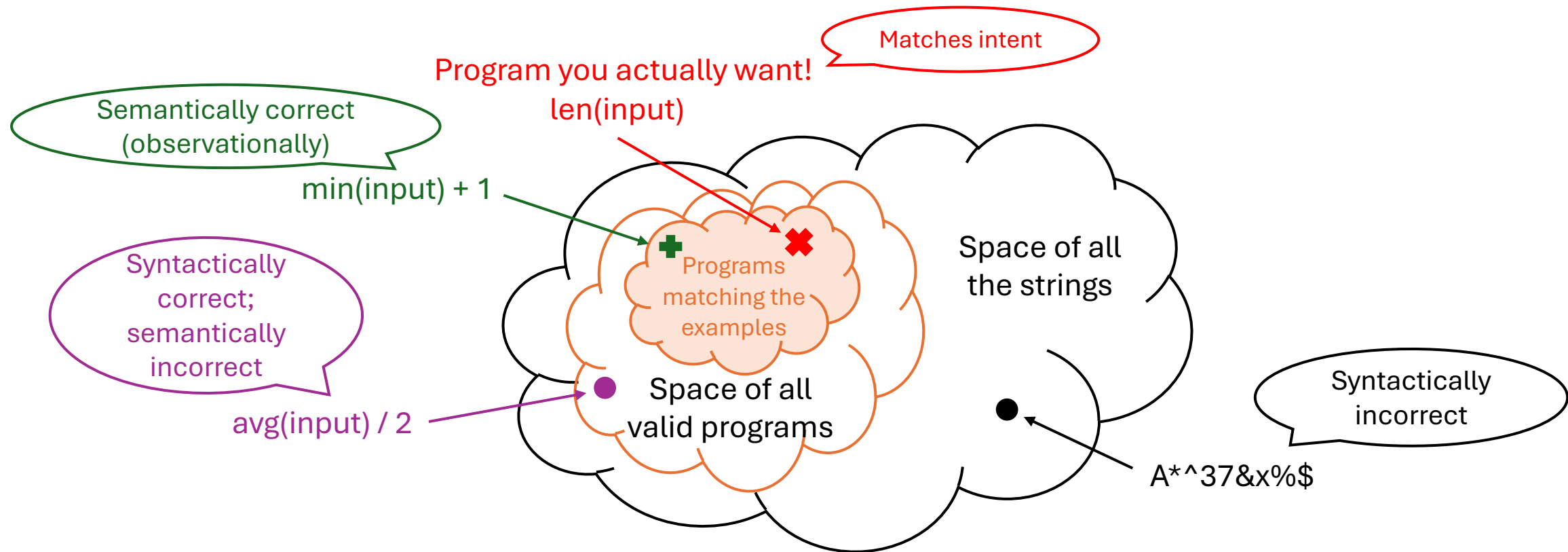
Semantics

```
def evaluate(program, input):  
    if isinstance(program, Empty):  
        return []  
    elif isinstance(program, Input):  
        return input  
    elif isinstance(program, Concat):  
        left = evaluate(program.left, input)  
        right = evaluate(program.right, input)  
        return left + right  
    elif ...
```

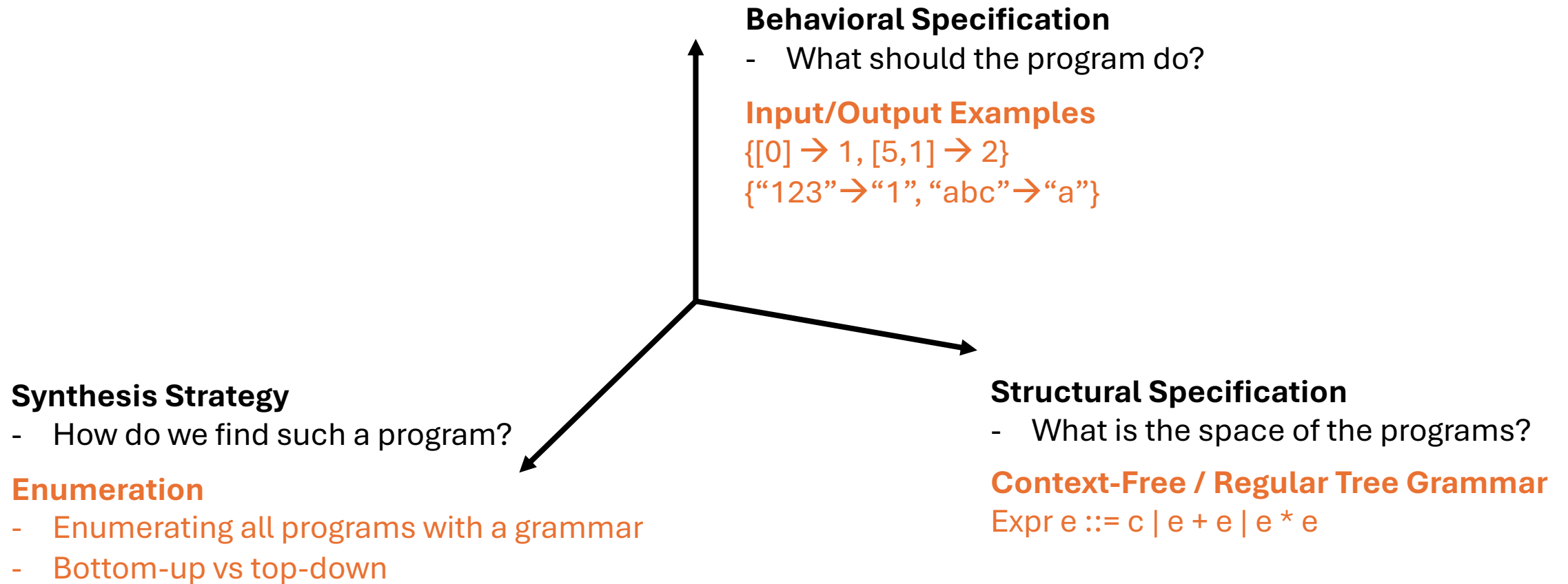
Examples

```
[0, 1]    -> 3  
[1]       -> 2  
[3, 5, 4] -> 4
```

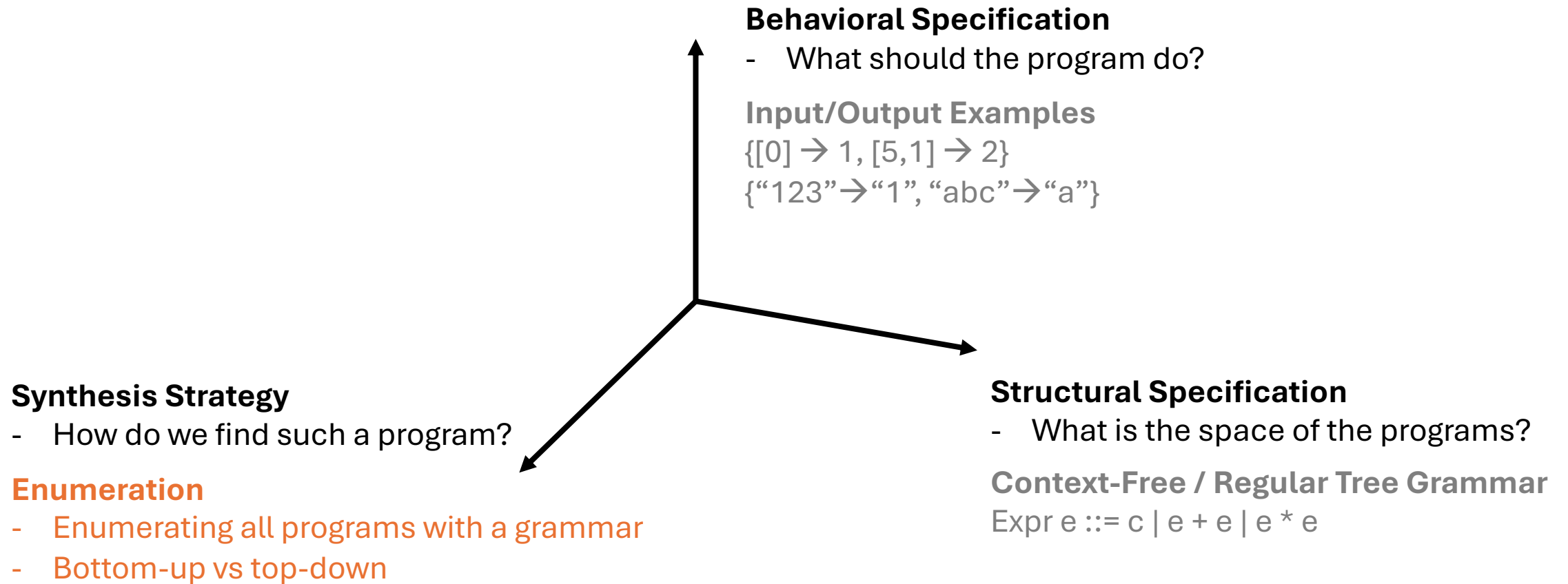
```
evaluate(`len(concat(single(3), input))`, [0, 1]) → len([3, 0, 1]) = 3  
└─> evaluate(`concat(single(3), input)`, [0, 1]) → [3] + [0, 1] = [3, 0, 1]  
    └─> evaluate(`single(3)`, [0, 1]) → [3]  
        └─> evaluate(`input`, [0, 1]) → [0, 1]
```



Today



Today



Enumerative search

- **Idea**: enumerate programs from the grammar one by one and test them on the examples
- **Challenge**: How do we systematically enumerate all programs?
 - Bottom-up
 - Top-down

Bottom-up enumeration

- Maintain a **bank** of complete programs
 - Starting from all the terminal symbols
- Combine programs in the bank using **production rules**
 - Applying all possible production rules at each iteration

```
(Program)  $\underline{P} ::= L \mid N$   
  (List)  $\underline{L} ::= \text{input}$   
            $\mid \text{empty}$   
            $\mid \text{single}(\underline{N})$   
            $\mid \text{concat}(\underline{L}, \underline{L})$   
  (Number)  $\underline{N} ::= \text{len}(\underline{L})$   
             $\mid \text{min}(\underline{L})$   
             $\mid \text{add}(\underline{N}, \underline{N})$   
             $\mid 0 \mid 1 \mid 2 \mid \dots$ 
```


Bottom-up enumeration: algorithm

```
bottom-up(< $\Sigma$ , N, R, S>, [ $i \rightarrow o$ ], max_depth):  
    bank := {}  
    for depth in [0..max_depth]:  
        forall rule in R:  
            forall new_prog in grow(rule, depth, bank):  
                if ( $A = S \wedge$  new_prog([ $i$ ]) = [ $o$ ]):  
                    return new_program  
            insert new_program to bank;  
  
grow( $A \rightarrow \sigma(A_1 \dots A_k)$ , d, bank):  
    if ( $d = 0 \wedge k = 0$ ) yield  $\sigma$  // terminal  
    else forall < $t_1, \dots, t_k$ > in bankk: // cartesian product  
        if  $A_i \rightarrow^* t_i$ :  
            yield  $\sigma(t_1, \dots, t_k)$ 
```

```
(Program)  $\underline{P} ::= L \mid N$   
(List)  $L ::=$  input  
           | empty  
           | single( $N$ )  
           | concat( $L, L$ )  
(Number)  $N ::=$  len( $L$ )  
              | min( $L$ )  
              | add( $N, N$ )  
              | 0 | 1 | 2 | ...
```

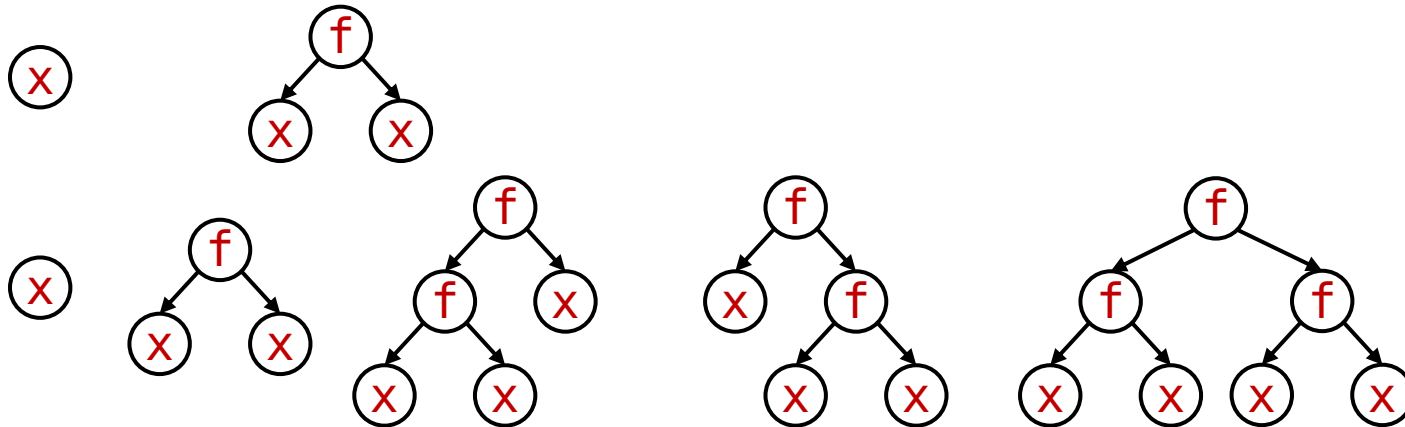
Bottom-up enumeration: example

$E ::= x \mid f(E, E)$

Depth ≤ 0 

Depth ≤ 1 

Depth ≤ 2 



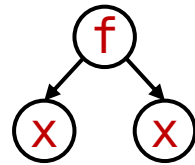
Bottom-up enumeration: example

$E ::= x \mid f(E, E)$

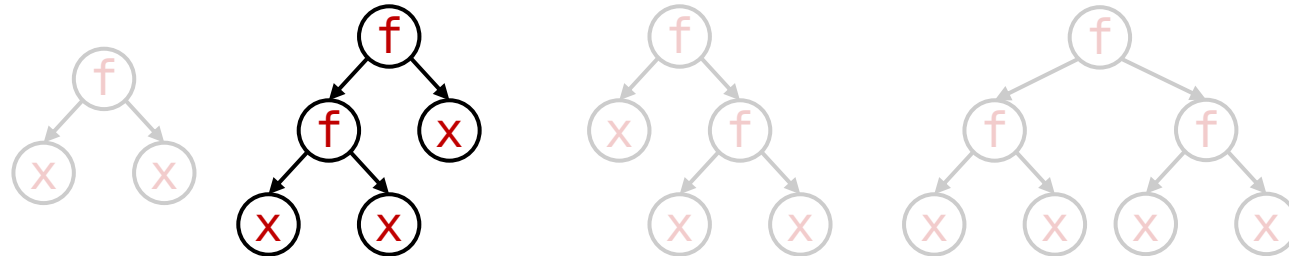
Depth ≤ 0



Depth ≤ 1

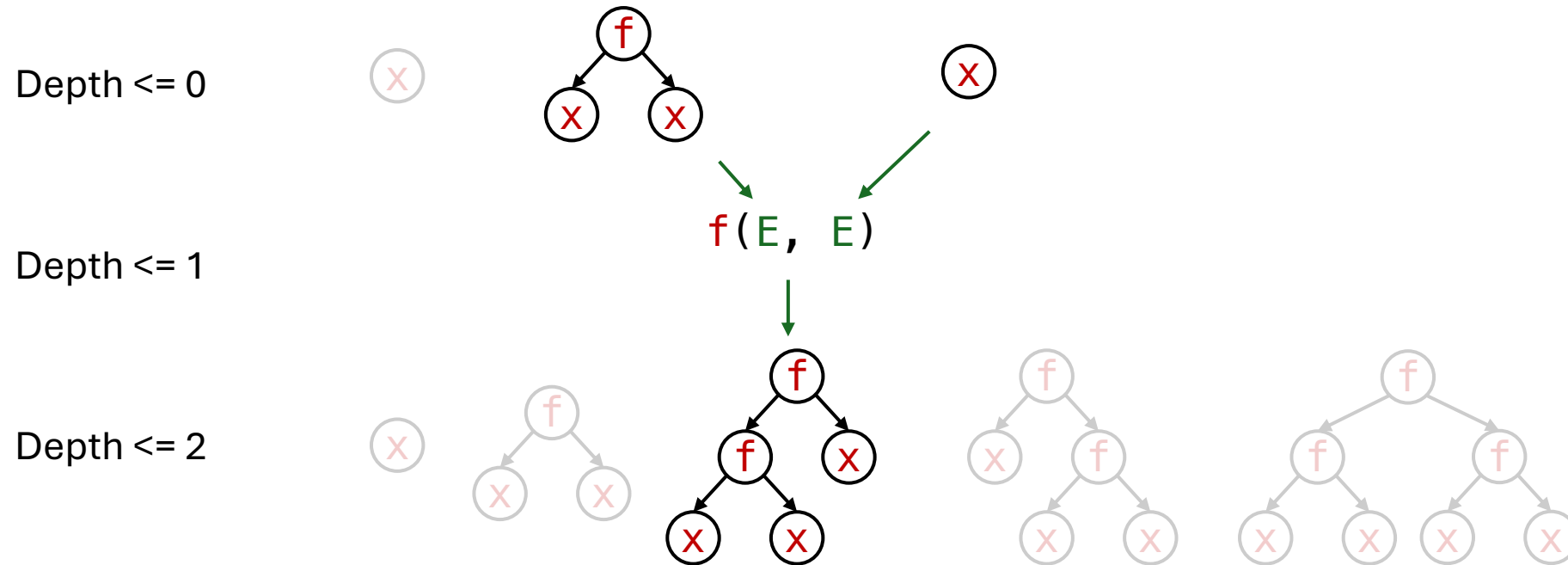


Depth ≤ 2



Bottom-up enumeration: example

$E ::= x \mid f(E, E)$

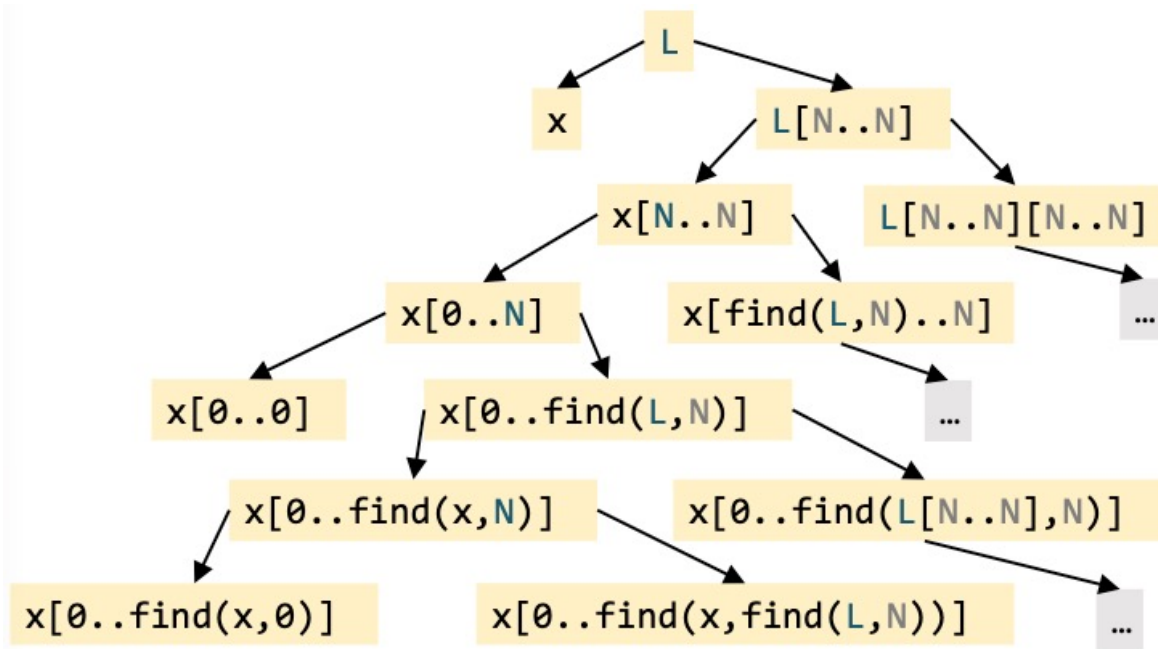


Top-down enumeration

- Search space is a tree, where:
 - Nodes are whole incomplete programs
 - Edges are derivations in a single step

```

(List)  L ::= L[N:N]
        | x // input
(Number) N ::= find(L, N)
        | 0
    
```

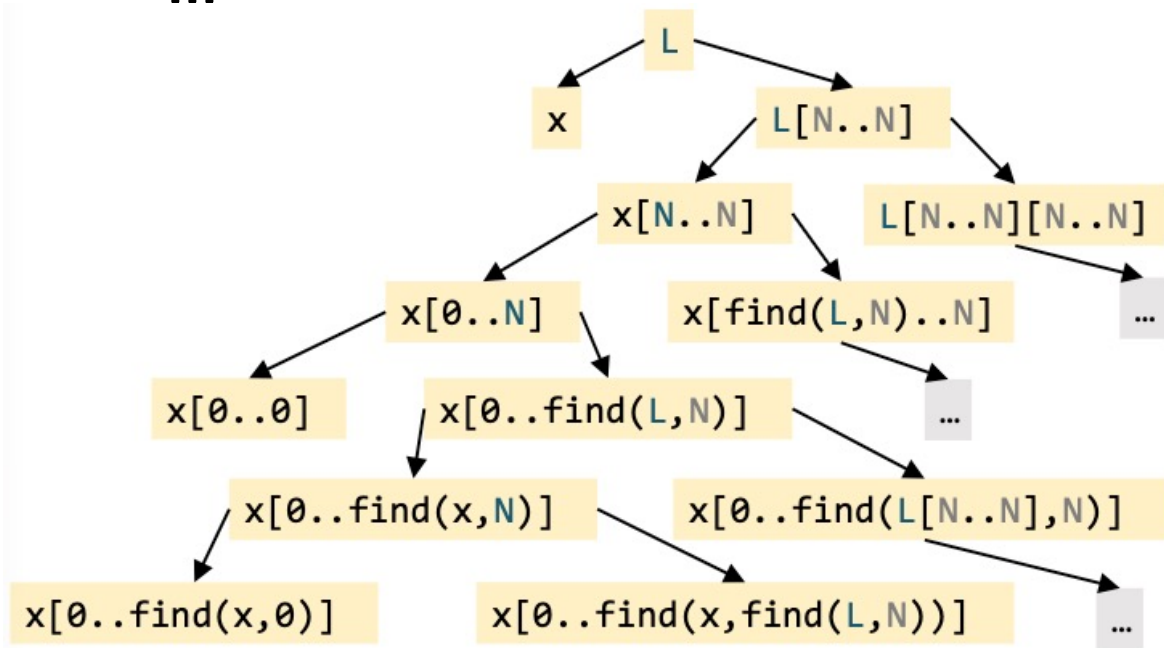


Top-down enumeration

- Search tree can be traversed
 - Depth-first
 - Breadth-first
 - ...

```

(List)  L ::= L[N:N]
        | x // input
(Number) N ::= find(L, N)
        | 0
    
```



Bottom-up vs top-down

Top-down

Program candidates are **whole**
but might not be **complete**

- Cannot always run on inputs
- Can always relate to outputs

Bottom-up

Program candidates are **complete**
but might not be **whole**

- Can always run on inputs
- Cannot always relate to outputs

How to make it scale

Prune

- Discard useless subprograms

Prioritize

- Explore more promising candidates

Summary

- Syntax
- Semantics
- Enumerative algorithms
 - Bottom-up
 - Top-down

Week 1

- Topic:
 - Programming language **syntax & semantics**
 - **Bottom-up enumerative synthesis** from examples
- Assignment 1:
 - Will be released by Friday
 - Due Thursday of the 3rd week (Sep 17)
 - Submission on GradeScope will be opened after release